南京大学 NANJING UNIVERSITY

机器学习导论-神经网络（续）

Introduction to Machine Learning-Neural Networks

李文斌

https://cs.nju.edu.cn/liwenbin

liwenbin@nju.edu.cn

2024年04月16日

大纲

- ❑ 背景引入
- ❑ CNN基础知识
- ❑ 深度学习技巧
- ❑ 深度学习工具箱
- ❑ 经典网络架构
- ❑ Transformer
- ❑ Mamba

背景

❑ 符号与术语（实数）

标量（scalar）：一个实数 $x \in \mathbb{R}$

向量（vector）：一个实数序列构成一个向量（粗斜体）$\boldsymbol{x} \in \mathbb{R}^d$

$$\boldsymbol{x} = (x_1, x_2, \ldots, x_d)^T$$

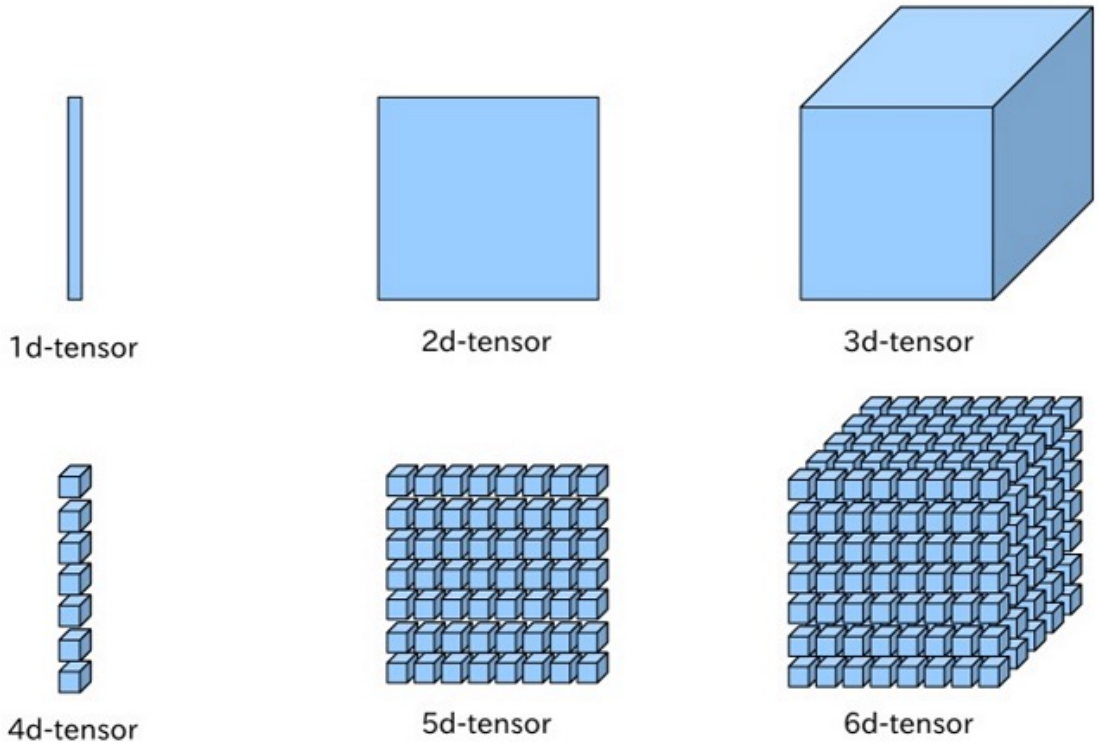
矩阵（matrix）：一个实数构成的矩形数组（大写粗体）$\boldsymbol{X} \in \mathbb{R}^{m*n}$

$$\boldsymbol{X} = \begin{bmatrix} x_{11} & \cdots & x_{1n} \\ \vdots & \ddots & \vdots \\ x_{m1} & \cdots & x_{mn} \end{bmatrix}$$

背景

❑ 符号与术语（实数）

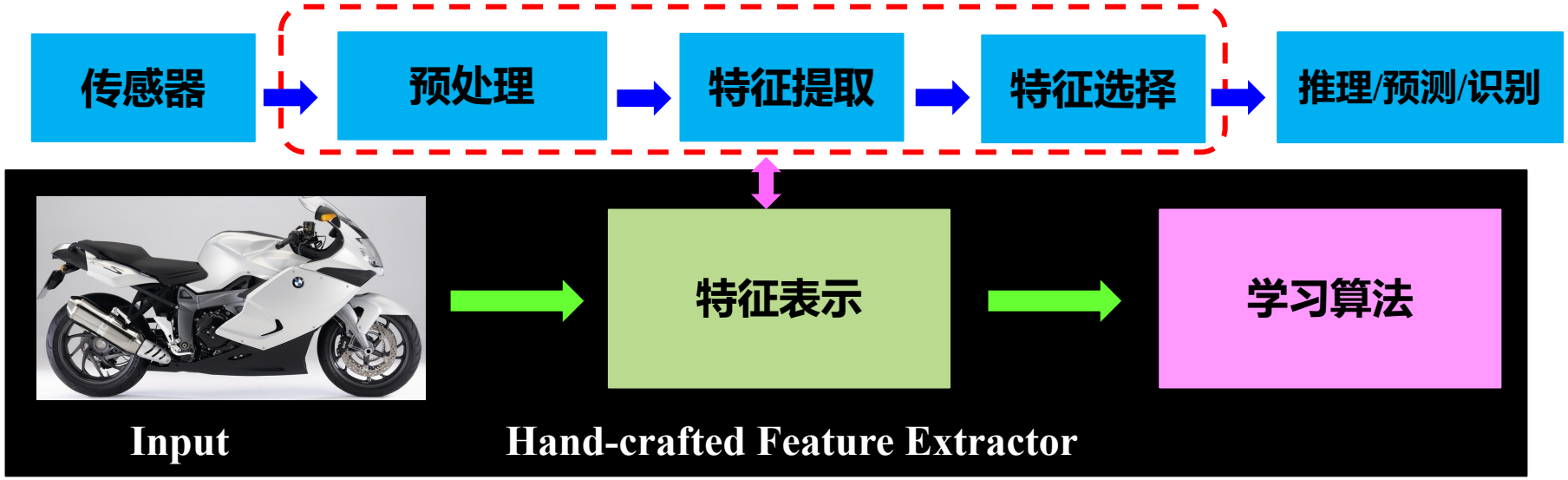
张量（tensor）：一个泛化的实数构成的n-维数组

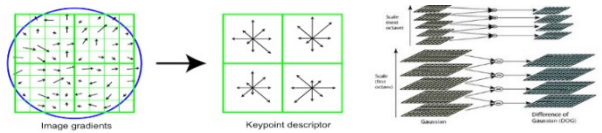


例子：$\boldsymbol{X} \in \mathbb{R}^{H*W*D}$ 是一个3阶张量

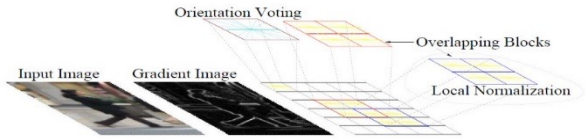
背景

❑ 模式识别的经典模型（浅层学习）

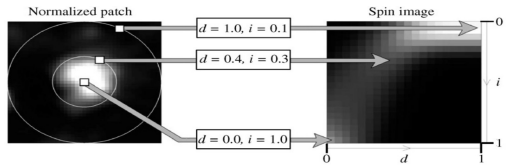




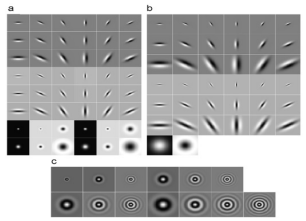
SIFT



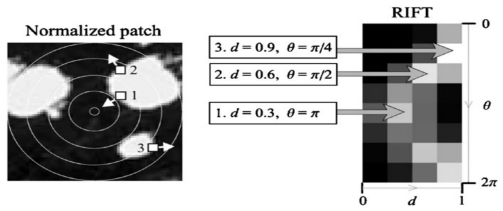
Hog



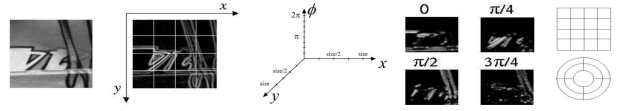
Spin image



Texton



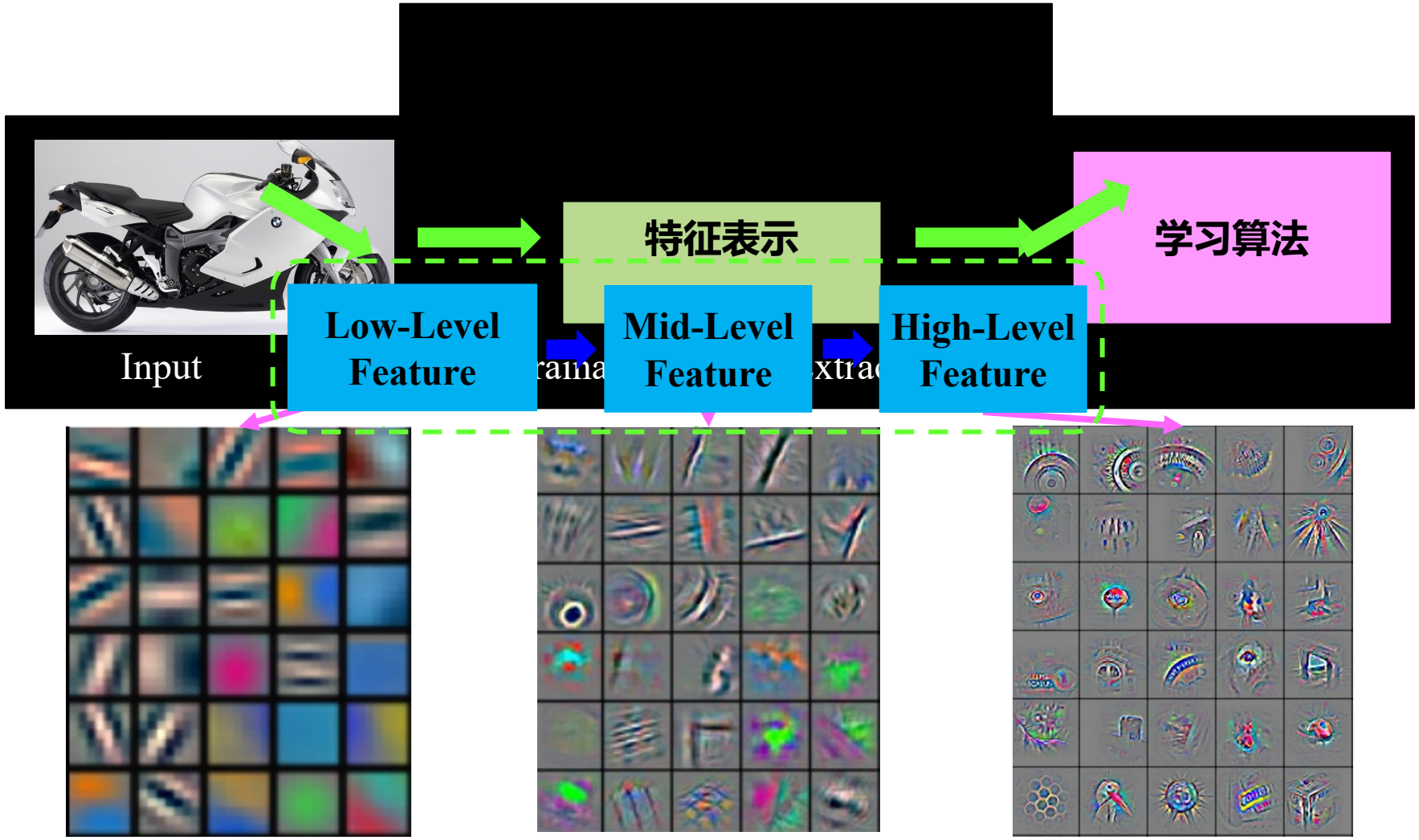
RIFT



GLOH

背景

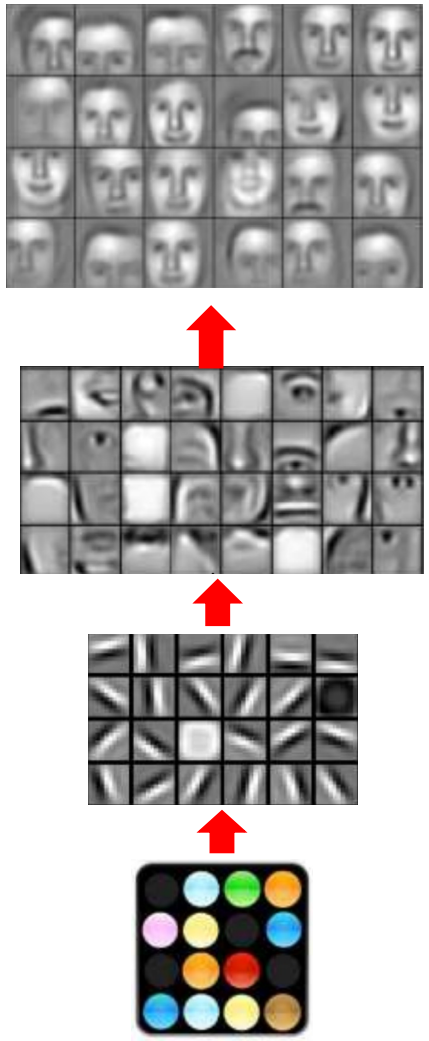
- 深度学习


Input
Low-Level Feature
Mid-Level Feature
High-Level Feature
特征表示
学习算法

背景

❑ 深度学习-学习层次特征

Sparse DBN: Trained on face images

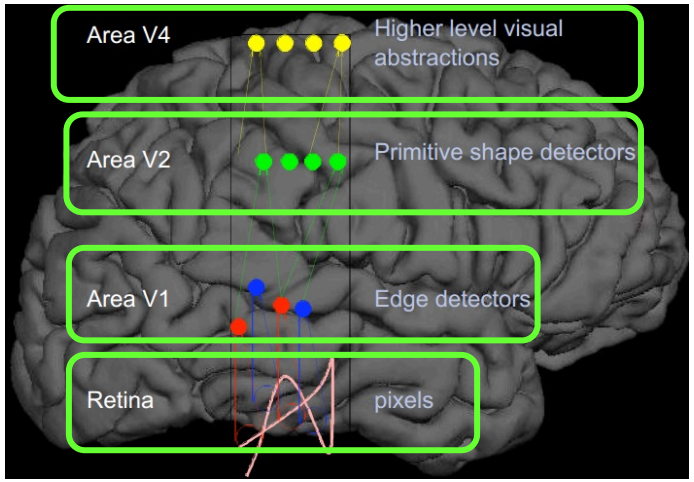


object models

object parts (combination edges)

edges

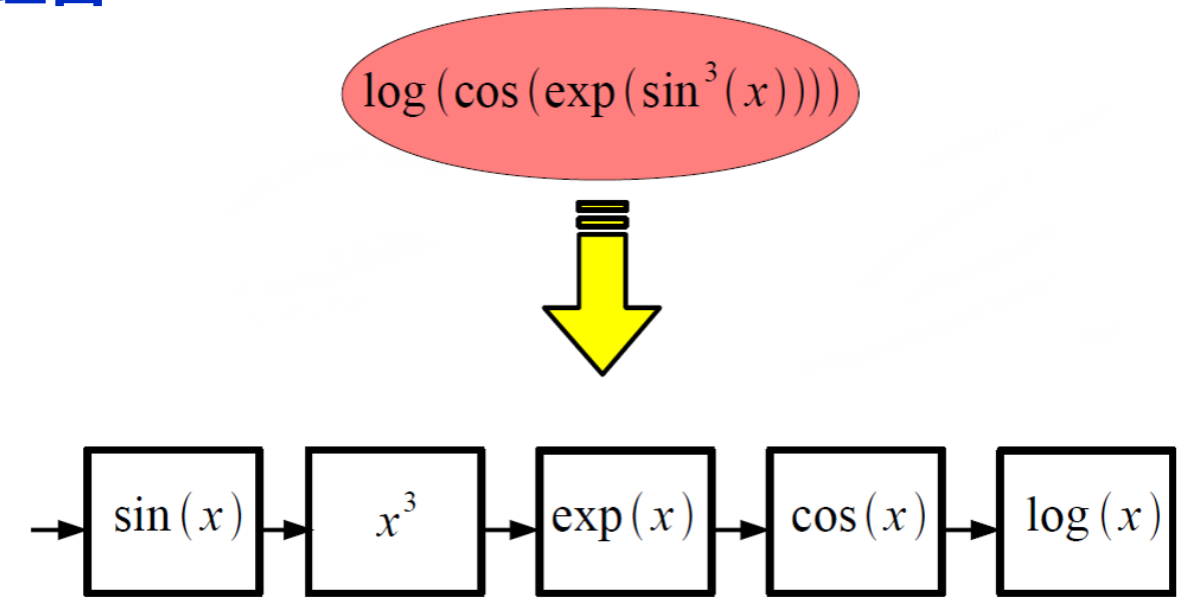
pixels



背景

❑ 深度学习-学习良好的非线性结构

✓ 例如：函数组合



- 通过联合简单的函数模块构建整个映射
- 每个简单的模块都有可学习的参数

背景

❑ 总结

手工特征

- 固定
- 难以设计
- 任务无关
- 低层特征
- 需要专业知识
- 分离的

深度特征

- 可学习的
- 黑盒
- 任务相关
- 学习层次表示和高级特征
- 专业解耦的
- 端到端的

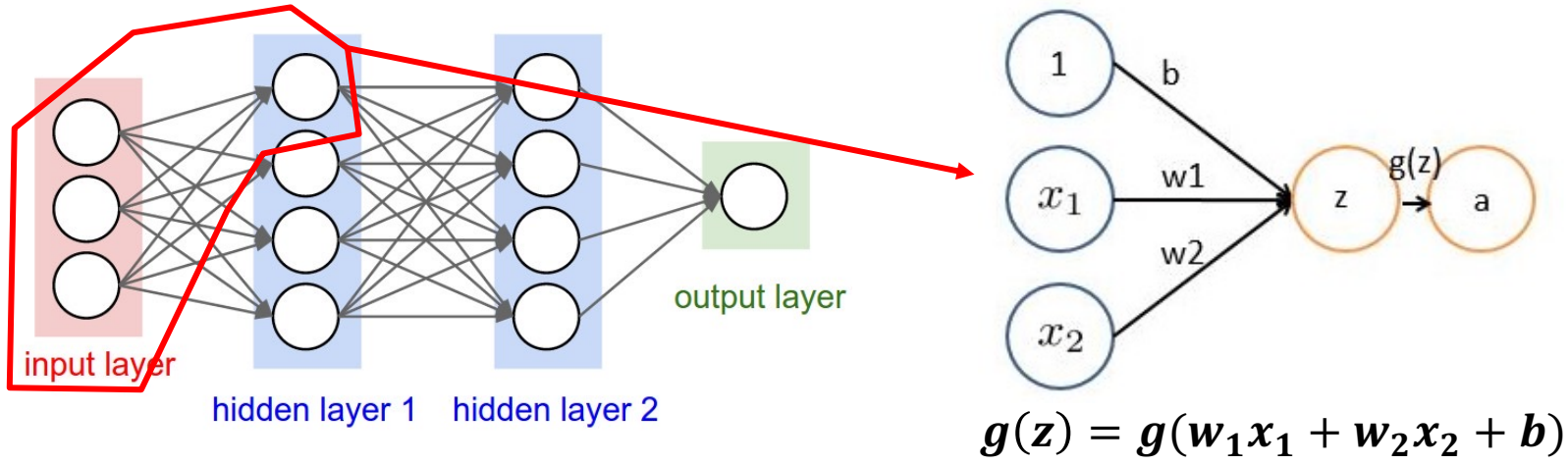
大纲

- 背景引入
- CNN基础知识
- 深度学习技巧
- 深度学习工具箱
- 经典网络架构
- Transformer
- Mamba

基本知识

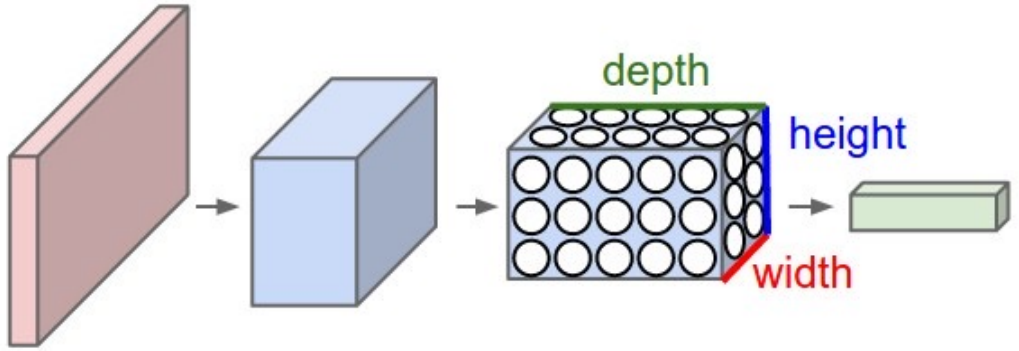
❑ 卷积Convolution

✓ 全连接神经网络



$$g(z) = g(w_1x_1 + w_2x_2 + b)$$

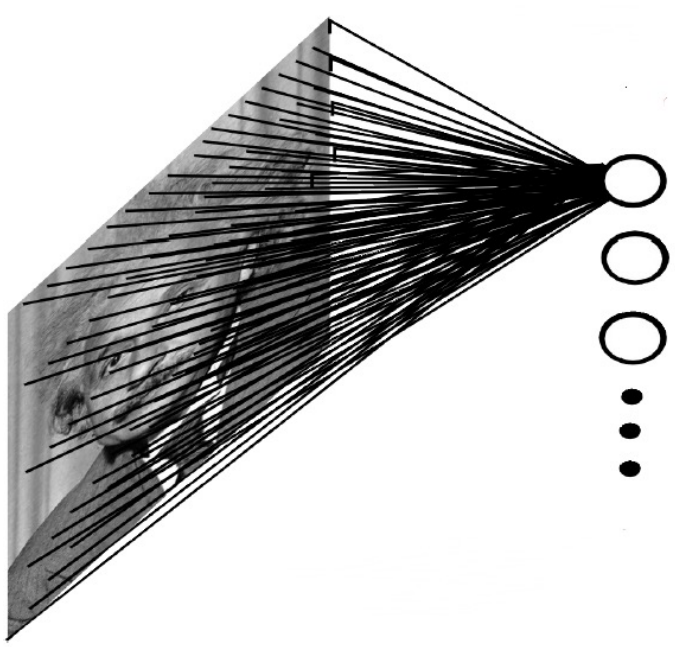
✓ 卷积神经网络



基本知识

❑ 卷积Convolution

✓ 完全连接



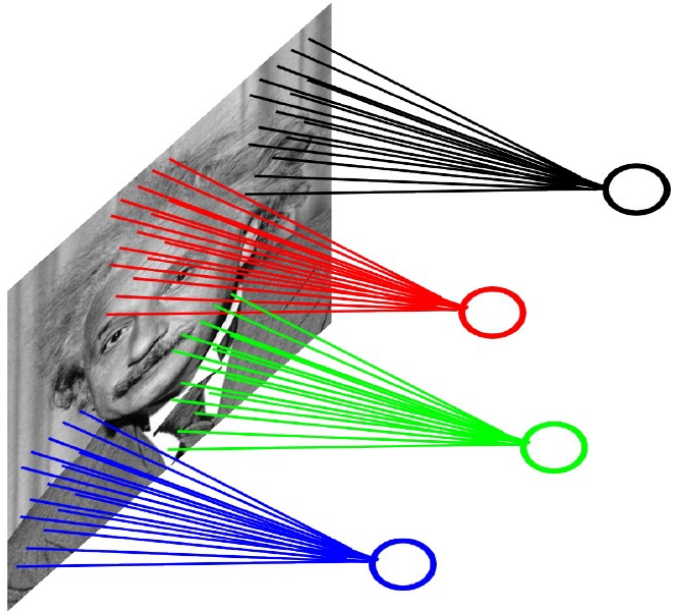
Example:
1000*1000 image
1M hidden units

10^{12} parameters

基本知识

❑ 卷积Convolution

✓ 局部连接



Example:

1000*1000 image

1M hidden units

Filter size: 10*10

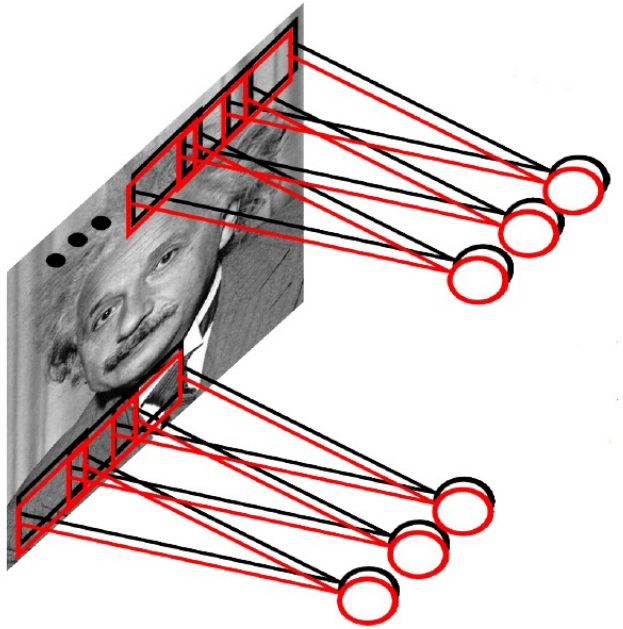
⇩

100M parameters

基本知识

❑ 卷积Convolution

✓ **使用共享参数和多个过滤器学习**



Example:

1000*1000 image

1M hidden units

Filter size: 10*10

100 Filters

10000 parameters

基本知识

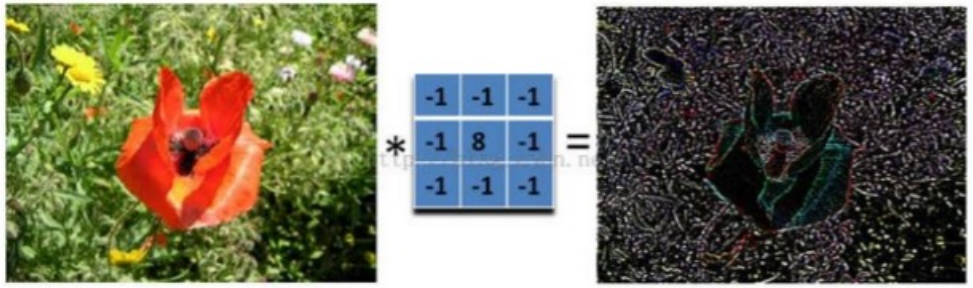
❑ 卷积Convolution

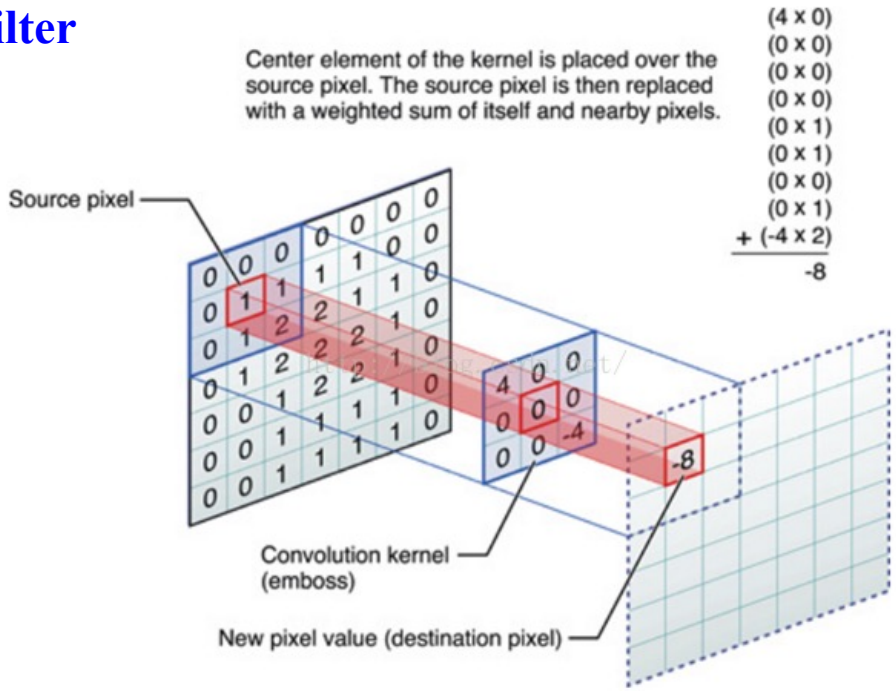
- 特征提取
- 超参数: depth, stride, zero-padding
- 过滤器: 3×3×3×96

Depth, the number of filters

Kernel size, local receptive fields

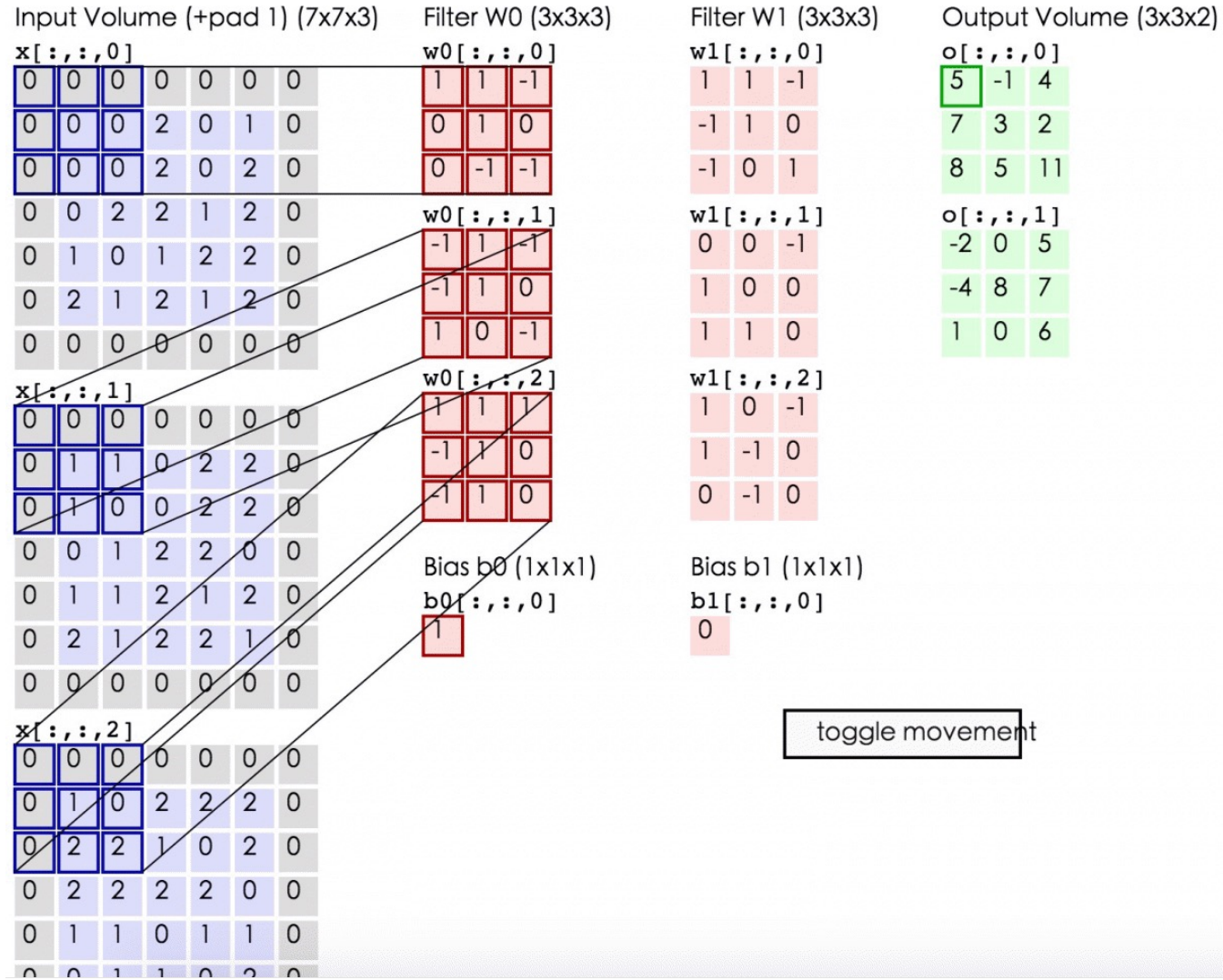
One filter





基本知识

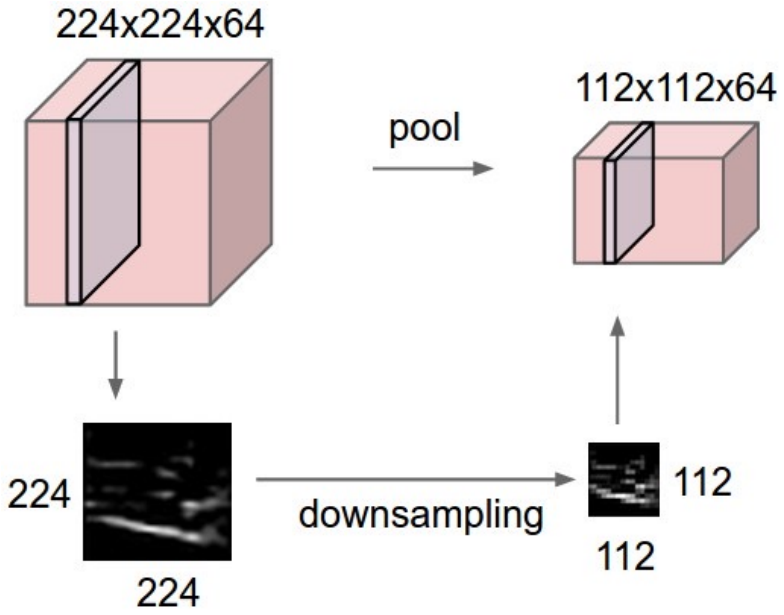
❑ 卷积Convolution



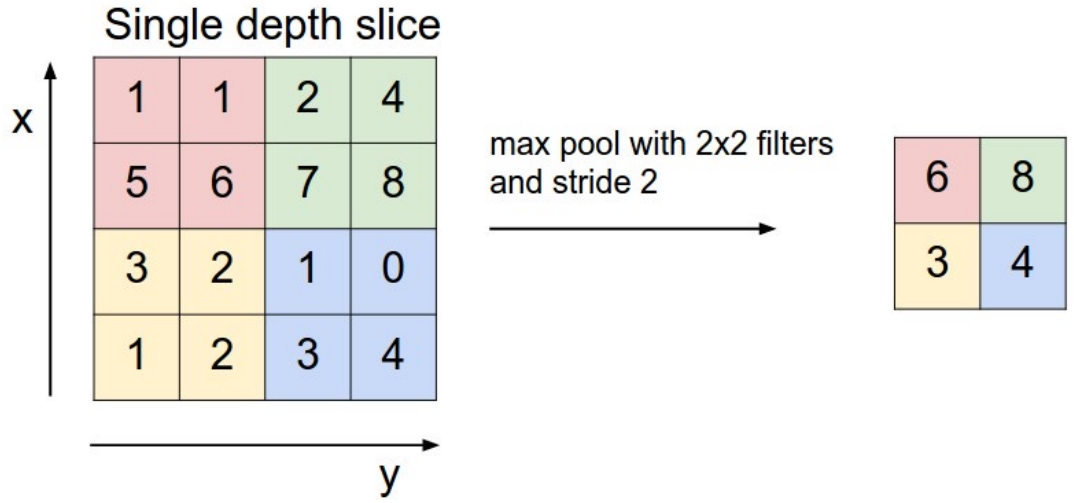
基本知识

❑ 池化Pooling/下采样Subsampling

- 减少参数
- 避免过拟合
- 扩大感受域



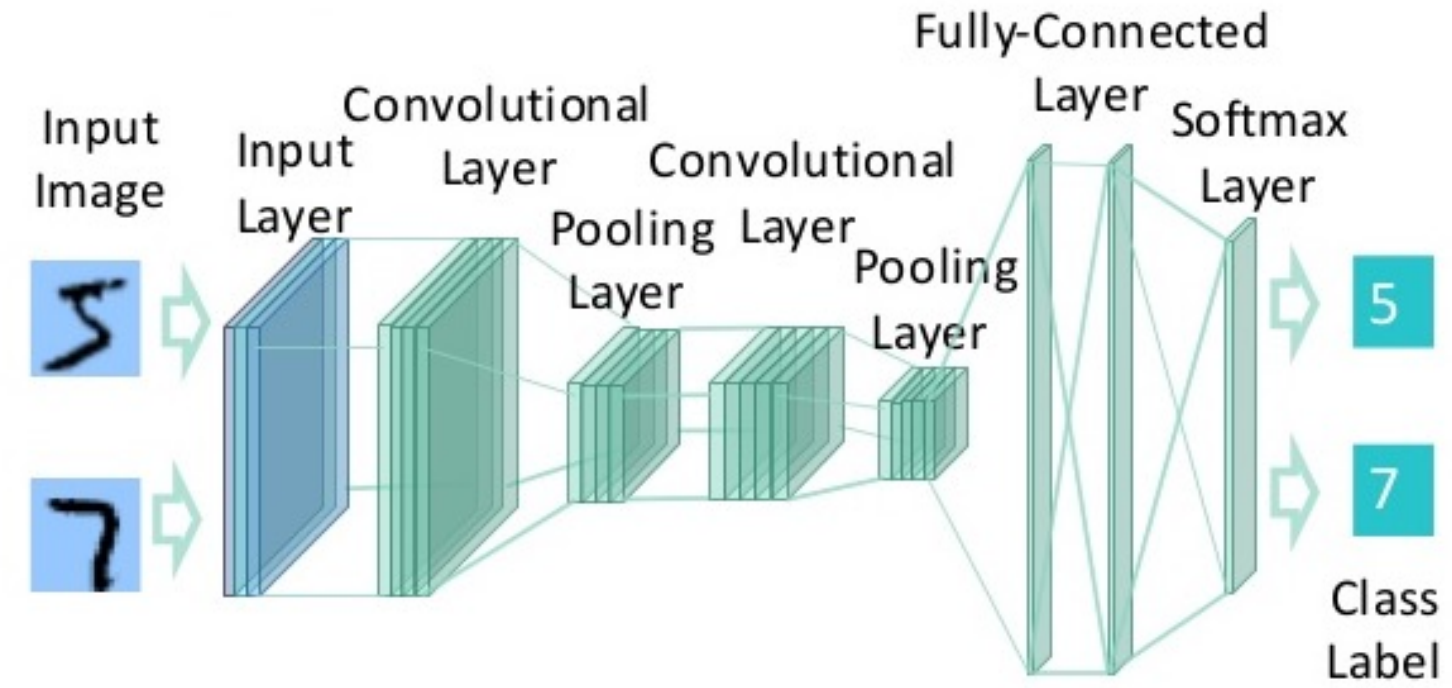
- Max pooling
- Average pooling
- L2-norm pooling
- Global average pooling
- Covariance pooling
- …..



基本知识

❑ 全连接层(Fully-Connected Layer)

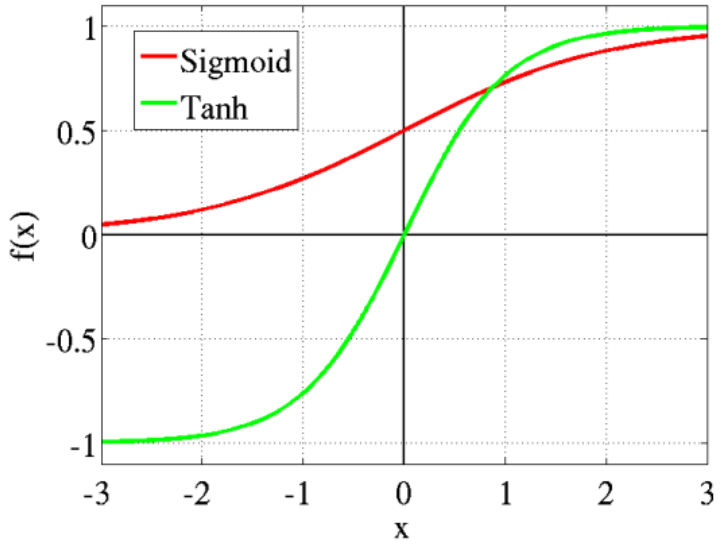
- ✓ FC layers: 全局特征抽取
- ✓ Softmax Layer: 分类层

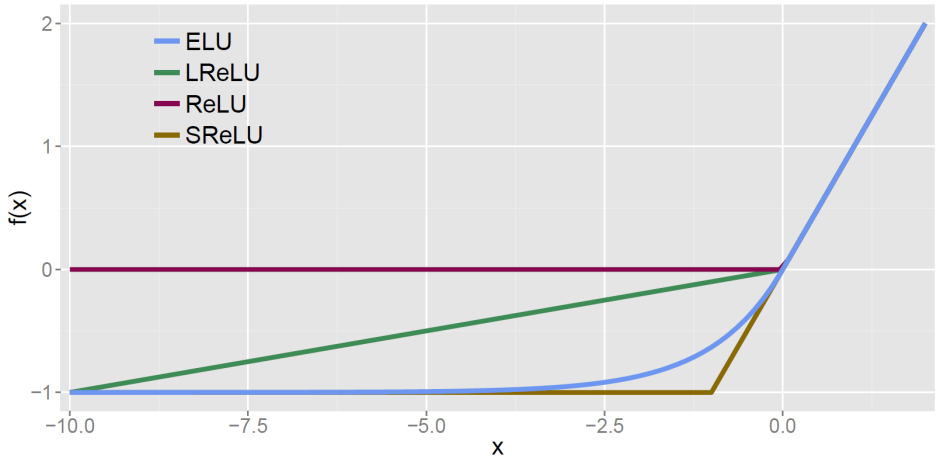


基本知识

❑ 激活

- **Sigmoid** $S(x) = \frac{1}{1+e^{-x}}$
- **tanh** $\tanh(x)$
- **ReLU** $\max(0, x)$
- **PReLU** $\max(ax, x)$
- **ELU** $y = \begin{cases} x & if\ x > 0 \\ a(e^x - 1)\ if\ x \leq 0 \end{cases}$
- **Maxout** $\max(w_1 x_1 + b_1, w_2 x_2 + b_2)$

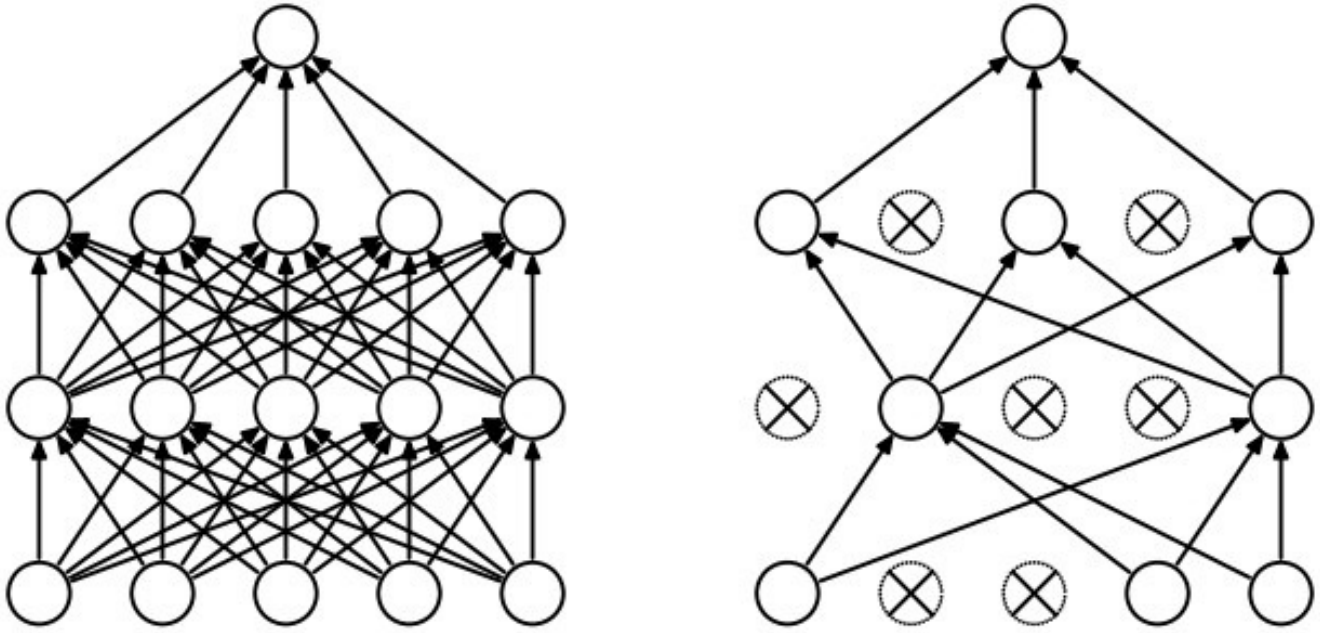




基本知识

❑ Dropout (Regularization)

- ✓ 一种对许多大的神经网络进行平均的有效方法，可以视为模型平均的一种形式
- ✓ 在最简单的情况下，每个神经单元都以固定的概率p保留，与其他单元无关

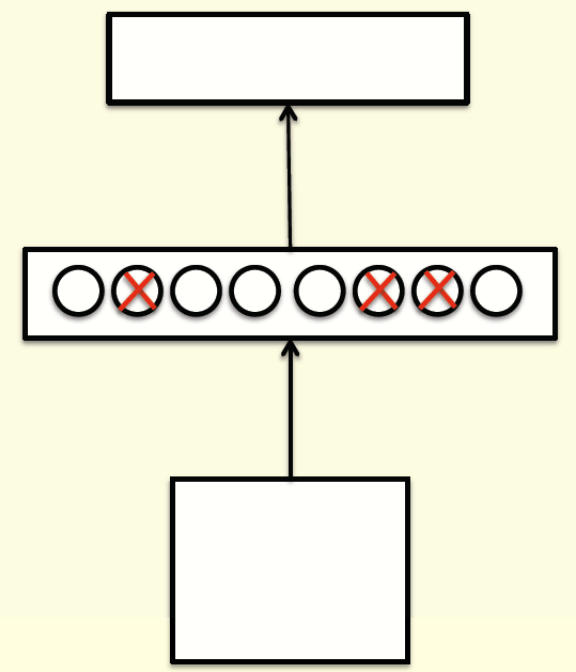


(a) Standard Neural Net　　(b) After applying dropout.

基本知识

❑ Dropout (Regularization)

- 考虑一个有一个隐藏层的神经网络
- 每次给出训练样本时，随机省略每个隐藏单元，概率为0.5
- 因此，从2^H个不同的架构中随机采样
  - 所有架构共享权重



- 从2^H模型中取样。因此，只有少数模型接受过训练，而且他们只得到了一个训练样本
- 权重的共享意味着每个模型都受到了非常强的正则化约束

基本知识

❑ Batch Normalization (Regularization)

✓ 目标：减少内部方差偏移或避免梯度扩散

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;
Parameters to be learned: γ, β
Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma\widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

$$\gamma^{(k)} = \sqrt{\mathrm{Var}[x^{(k)}]}$$

$$\beta^{(k)} = \mathrm{E}[x^{(k)}]$$

$$\Downarrow$$

$$y_i = x_i$$

Blog: http://blog.csdn.net/hjimce/article/details/50866313
Zhihu: https://www.zhihu.com/question/38102762
Paper: Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift

基本知识

❑ 梯度(SGD)

An abstract description of the CNN structure

$$x^1 \longrightarrow \boxed{w^1} \longrightarrow x^2 \longrightarrow \cdots \longrightarrow x^{L-1} \longrightarrow \boxed{w^{L-1}} \longrightarrow x^L \longrightarrow \boxed{w^L} \longrightarrow z$$

For example, a simple loss function:

$$z = \frac{1}{2}\|\boldsymbol{t} - \boldsymbol{x}^L\|^2 \Rightarrow \begin{array}{l} \dfrac{\partial z}{\partial \boldsymbol{w}^L} \text{ is empty} \\ \dfrac{\partial z}{\partial \boldsymbol{x}^L} = \boldsymbol{x}^L - \boldsymbol{t} \end{array}$$

According to chain rule, we can compute $\frac{\partial z}{\partial \boldsymbol{w}^i}$ and $\frac{\partial z}{\partial \boldsymbol{x}^i}$

$$\cdots x^i \rightarrow \boxed{w^i} \rightarrow x^{i+1} \cdots$$

$$\frac{\partial z}{\partial (\mathrm{vec}(\boldsymbol{w}^i)^T)} = \frac{\partial z}{\partial (\mathrm{vec}(\boldsymbol{x}^{i+1})^T)} \frac{\partial \,\mathrm{vec}(\boldsymbol{x}^{i+1})}{\partial (\mathrm{vec}(\boldsymbol{w}^i)^T)}$$

$$\frac{\partial z}{\partial (\mathrm{vec}(\boldsymbol{x}^i)^T)} = \frac{\partial z}{\partial (\mathrm{vec}(\boldsymbol{x}^{i+1})^T)} \frac{\partial \,\mathrm{vec}(\boldsymbol{x}^{i+1})}{\partial (\mathrm{vec}(\boldsymbol{x}^i)^T)}$$

基本知识

❑ 梯度(SGD)

❑ 最小化一个损失函数

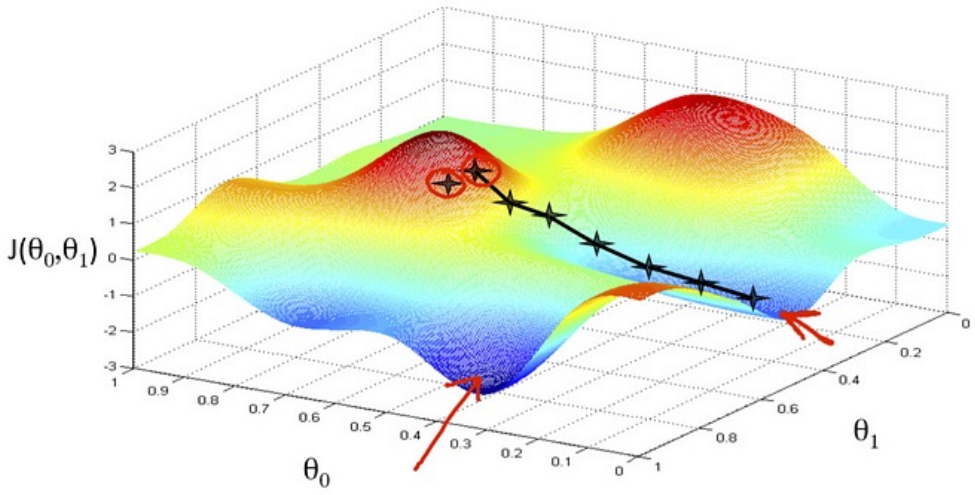
$$J(\theta) = \frac{1}{2}\frac{1}{m}\sum_{i=1}^{m}(f_\theta(x) - y)^2,$$

$$f_\theta(x) = \sum_{j=0}^{n}\theta_j x_j = \theta_0 + \theta_1 x_1 + \cdots + \theta_n x_n$$

❑ 梯度下降（gradient descent）

$$\theta_j := \theta_j - \alpha\frac{\partial J(\theta)}{\partial \theta_j}$$

$$\begin{aligned}\frac{\partial J(\theta)}{\partial \theta_j} &= 2\cdot\frac{1}{2}(f_\theta(x) - y)\cdot\frac{\partial(f_\theta(x) - y)}{\partial \theta_j} \\ &= 2\cdot\frac{1}{2}(f_\theta(x) - y)\cdot\frac{\partial(\sum_{i=0}^{n}\theta_i x_i - y)}{\partial \theta_j} \\ &= (f_\theta(x) - y)\cdot x_j\end{aligned}$$



基本知识

❑ 梯度(SGD)

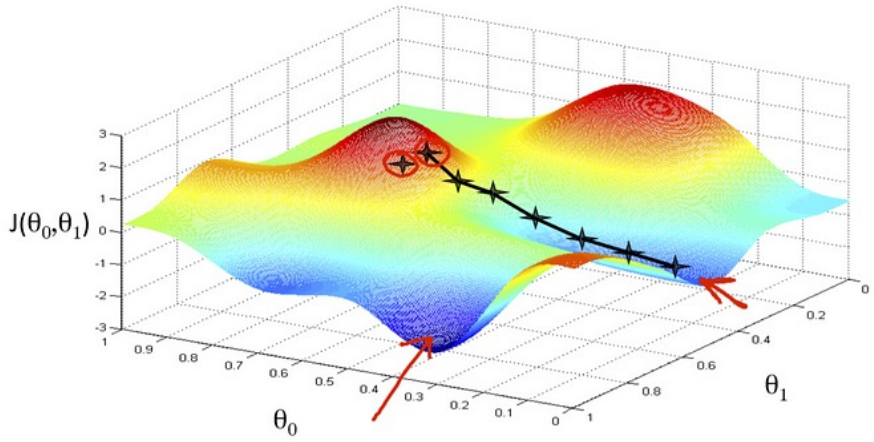
❑ 最小化一个损失函数

$$J(\theta) = \frac{1}{2}\frac{1}{m}\sum_{i=1}^{m}(f_\theta(x) - y)^2,$$

$$f_\theta(x) = \sum_{j=0}^{n}\theta_j x_j = \theta_0 + \theta_1 x_1 + \cdots + \theta_n x_n$$

❑ 批量梯度下降（gradient descent)

$$\theta_j := \theta_j - \alpha\frac{1}{m}\sum_{i=1}^{m}\left(f_\theta\left(x^{(i)}\right) - y^{(i)}\right)\cdot x_j^{(i)}$$



基本知识

❑ 梯度(SGD)

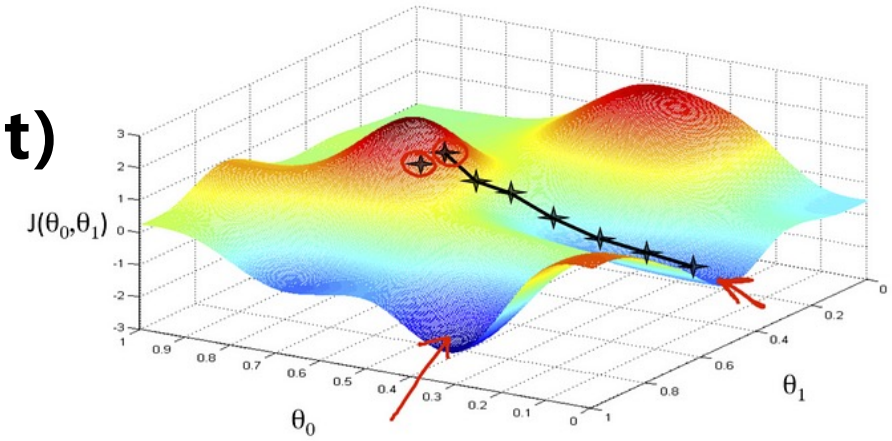
❑ 最小化一个损失函数

$$J(\theta) = \frac{1}{2}\frac{1}{m}\sum_{i=1}^{m}(f_\theta(x) - y)^2,$$

$$f_\theta(x) = \sum_{j=0}^{n}\theta_j x_j = \theta_0 + \theta_1 x_1 + \cdots + \theta_n x_n$$

❑ 随机梯度下降（stochastic gradient descent）

$$\theta_j := \theta_j - \alpha\left(f_\theta\left(x^{(i)}\right) - y^{(i)}\right)\cdot x_j^{(i)}$$



基本知识

❑ 梯度(SGD)

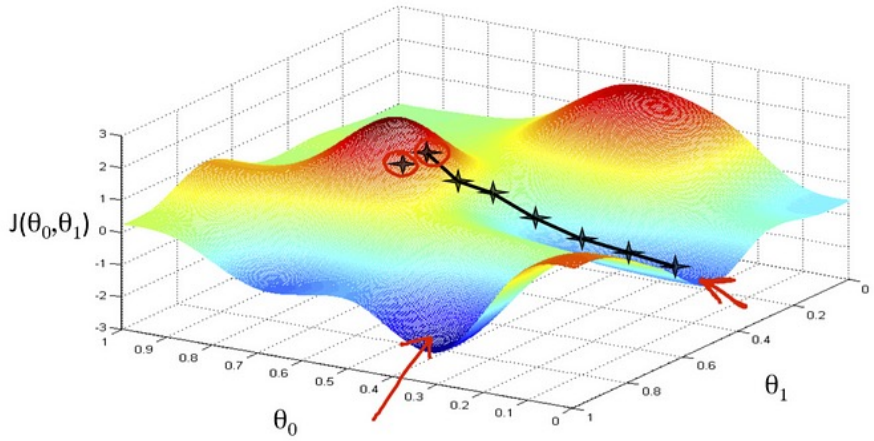
❑ 最小化一个损失函数

$$J(\theta) = \frac{1}{2}\frac{1}{m}\sum_{i=1}^{m}(f_\theta(x) - y)^2,$$

$$f_\theta(x) = \sum_{j=0}^{n}\theta_j x_j = \theta_0 + \theta_1 x_1 + \cdots + \theta_n x_n$$

❑ Mini-batch随机梯度下降

$$\theta_j := \theta_j - \alpha\frac{1}{b}\sum_{i=1}^{b}\left(f_\theta\left(x^{(i)}\right) - y^{(i)}\right)\cdot x_j^{(i)}$$



基本知识

❑ 交叉熵损失函数(Cross-entropy loss)

✓ 形式化

$$L_i = -\log\left(\frac{e^{f_{y_i}}}{\sum_j e^{f_j}}\right) \quad \text{Or} \quad L_i = -f_{y_i} + \log\sum_j e^{f_j}$$

Softmax function

基本知识

❑ 交叉熵损失

$$L_i = -\log\left(\frac{e^{f_{y_i}}}{\sum_j e^{f_j}}\right)$$

✓ 从信息论的角度

交叉熵: $H(p,q) = -\sum_x p(x)\log q(x)$

真实的概率分布:
$p = [0, \ldots, 1, \ldots, 0]$

预测的概率分布:
$q = \frac{e^{f_{y_i}}}{\sum_j f_j}$

$$H(p,q) = H(p) + D_{KL}(p||q)$$

$H(p)$: **0**

$D_{KL}(p||q)$: **Relative Entropy**

基本知识

❑ 交叉熵损失

$$L_i = -\log\left(\frac{e^{f_{y_i}}}{\sum_j e^{f_j}}\right)$$

✓ 从概率的角度

$$P(y_i \mid x_i; W) = \frac{e^{f_{y_i}}}{\sum_j e^{f_j}}$$

- 将错误类别的负对数概率降至最低
- 最大化真实类别的正对数概率

基本知识

❑ 损失函数（回归损失）

✓ 形式化

L2-Norm: $$L_i = \|f - y_i\|_2^2$$

L1-Norm: $$L_i = \|f - y_i\|_1 = \sum_j | f_j - (y_i)_j|$$

基本知识

❑ 损失函数（三元组损失Triplet loss ）

✓ 形式化

$$E(W') = \sum_{(a,p,n)\in T} \max\{0, \alpha - \|\mathbf{x}_a - \mathbf{x}_n\|_2^2 + \|\mathbf{x}_a - \mathbf{x}_p\|_2^2\}$$

- 一个三元组Triplet$(\boldsymbol{x}_a, \boldsymbol{x}_n, \boldsymbol{x}_p)$
- $\boldsymbol{a \geq 0}$ 是一个固定的标量，即间隔（Margin）
- **T** 是三元组的总个数
- $\boldsymbol{\max(0, z)}$ 是铰链损失Hinge loss

大纲

- ❑ 背景引入
- ❑ CNN基础知识
- ❑ 深度学习技巧
- ❑ 深度学习工具箱
- ❑ 经典网络架构
- ❑ Transformer
- ❑ Mamba

深度学习技巧

❑ 深度学习

- ✓ 数据增广（Data Augmentation）
- ✓ 预处理（Pre-Processing）
- ✓ 初始化（Initialization）
- ✓ 过滤器（Filters）
- ✓ 池化大小（Pooling size）
- ✓ 学习率（Learning rate）

深度学习技巧

❑ 深度学习

✓ 数据增广（Data Augmentation）

- Horizontally flipping, random crops and color jittering, etc.

✓ 预处理 （Pre-Processing）

- Zero-center and normalize, PCA Whitening, etc.

✓ 初始化 （Initialization）

- Small random numbers, Calibrating the variances, etc.

深度学习技巧

❑ 深度学习

✓ 过滤器 （Filters）

- Batch size: power-of-2 (32, 64, 224, 384…)
- Small filter (e.g., 3×3 or 1×1) and small stride (e.g., 1)

✓ 池化大小（Pooling size）

- Zero-padding, pooling size (e.g., 2×2)

✓ 学习率 （Learning rate）

- Step attenuation, Exponential level attenuation, etc.
- Choose little learning rates when fine-tuning

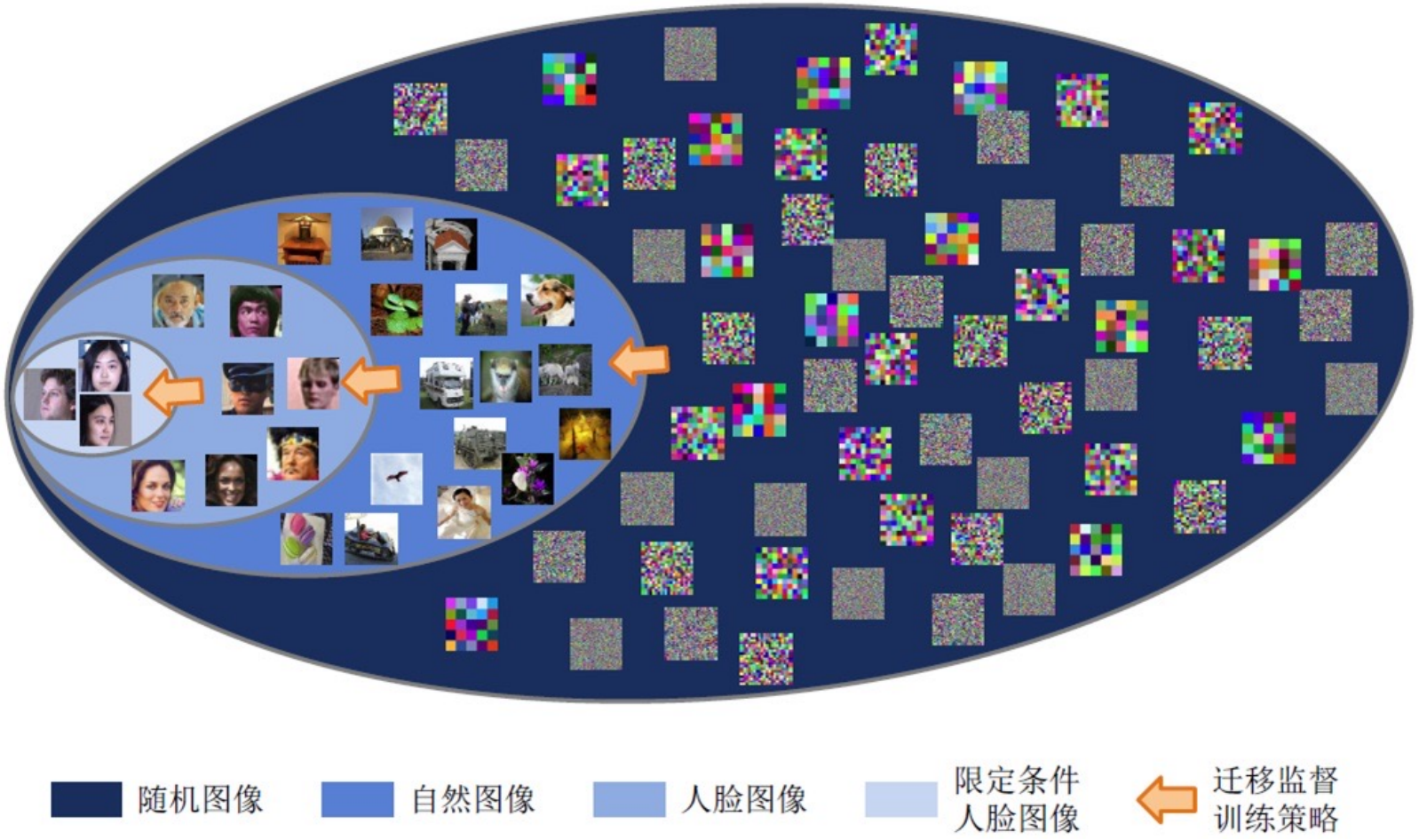
深度学习技巧

- 在预训练模型上微调

	very similar dataset	very different dataset
very little data	Use linear classifier on top layer	You're in trouble... Try linear classifier from different stages
quite a lot of data	Finetune a few layers	Finetune a large number of layers

深度学习技巧

- 在预训练模型上微调



深度学习技巧

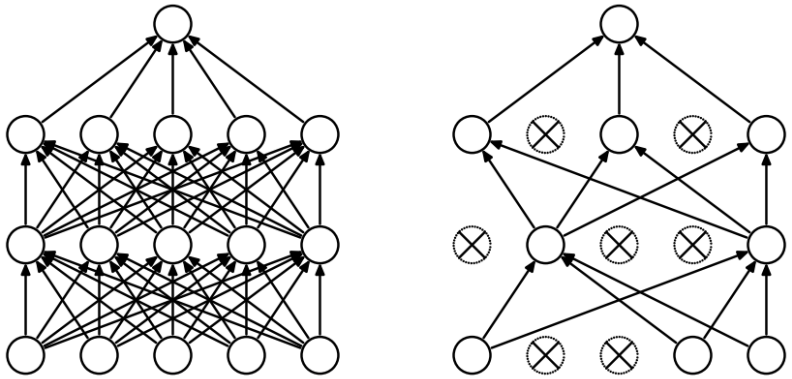
❑ 激活函数与正则化

✓ 激活函数（Activation Functions）

- ReLU, PReLU and RReLU

✓ 正则化（Regularizations）

- L2 regularization (i.e., $\frac{1}{2}\lambda w^2$)
- L1 regularization (i.e., $\lambda|\mathrm{w}|$)
- Max norm constraints (i.e., $\|w\|_2 < c$)
- Dropout
- Batch Normalization



(a) Standard Neural Net　　(b) After applying dropout.

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;
Parameters to be learned: γ, β
Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

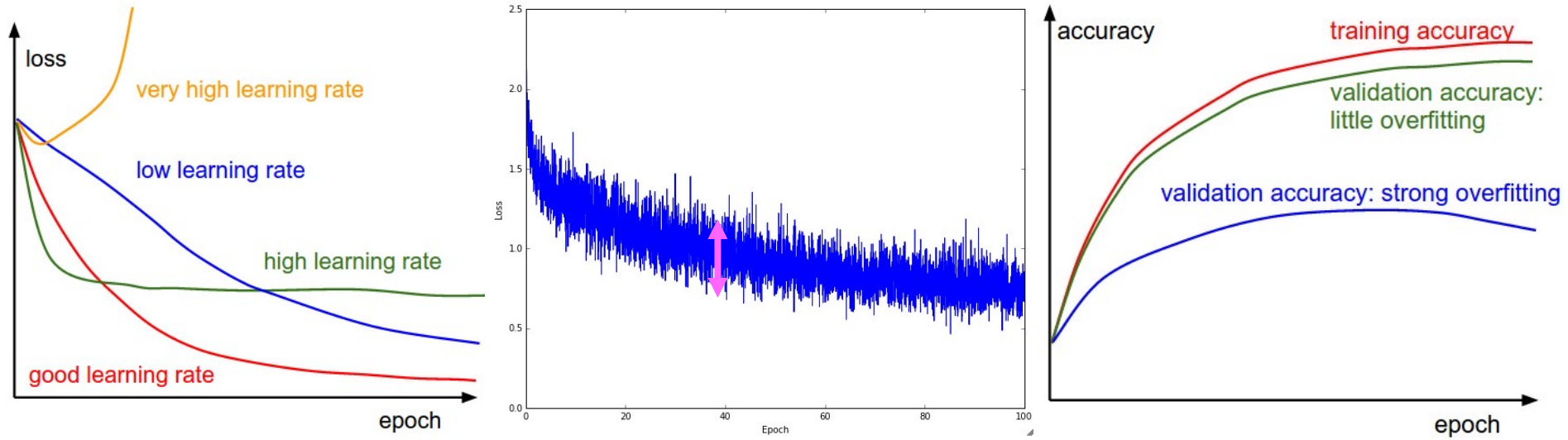
$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma\widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

深度学习技巧

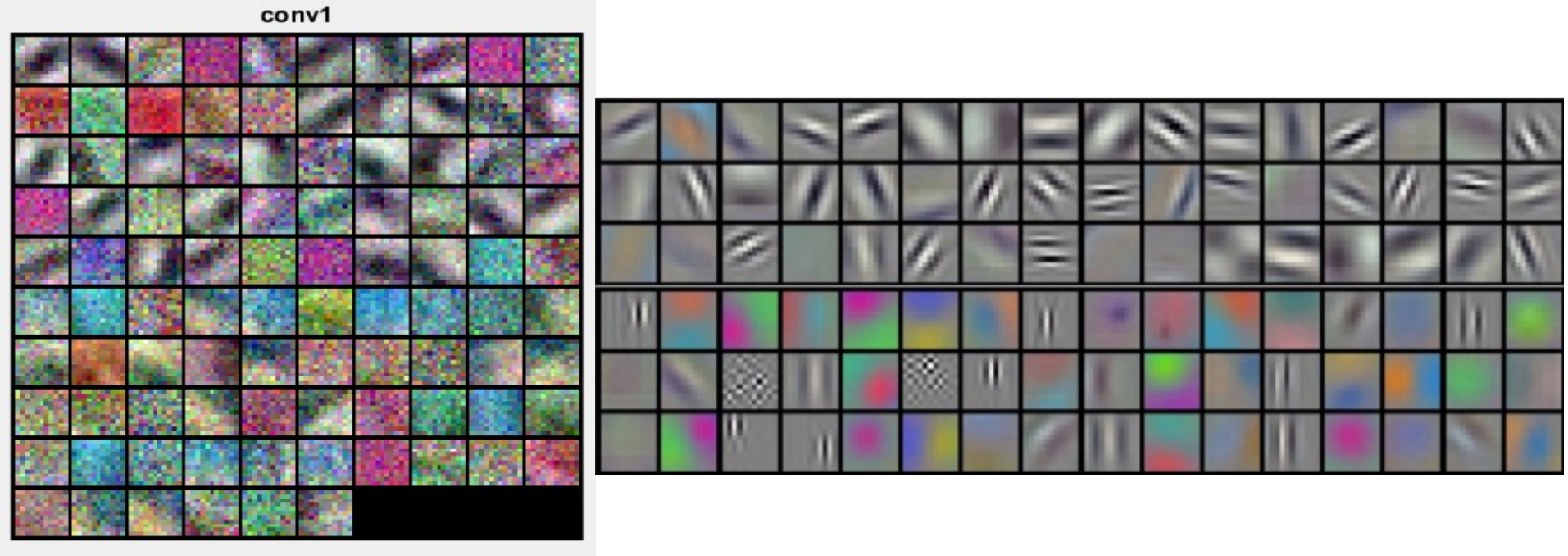
❑ 图像分析（Insights from figures）



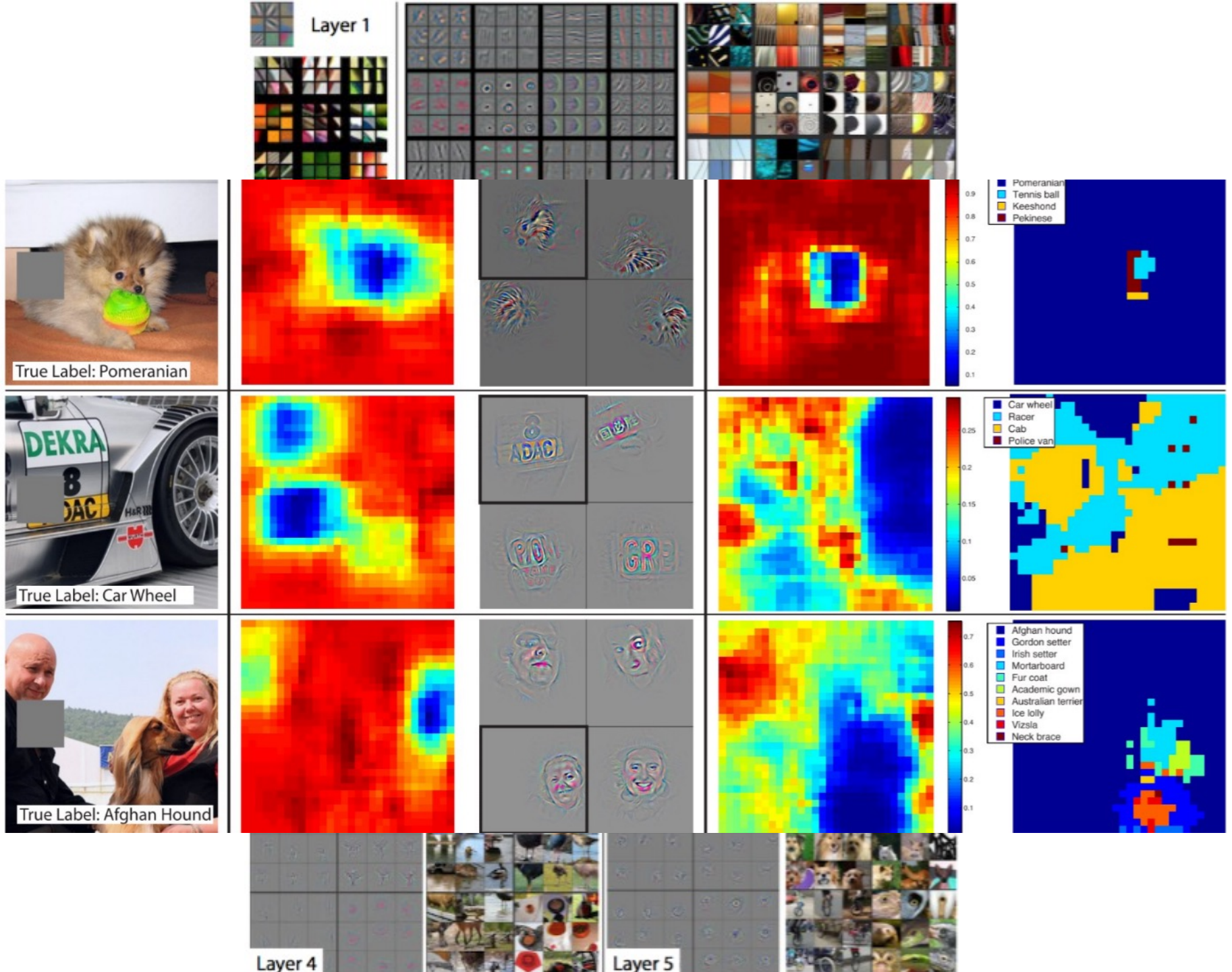
深度学习技巧

❑ 可视化（Visualization）

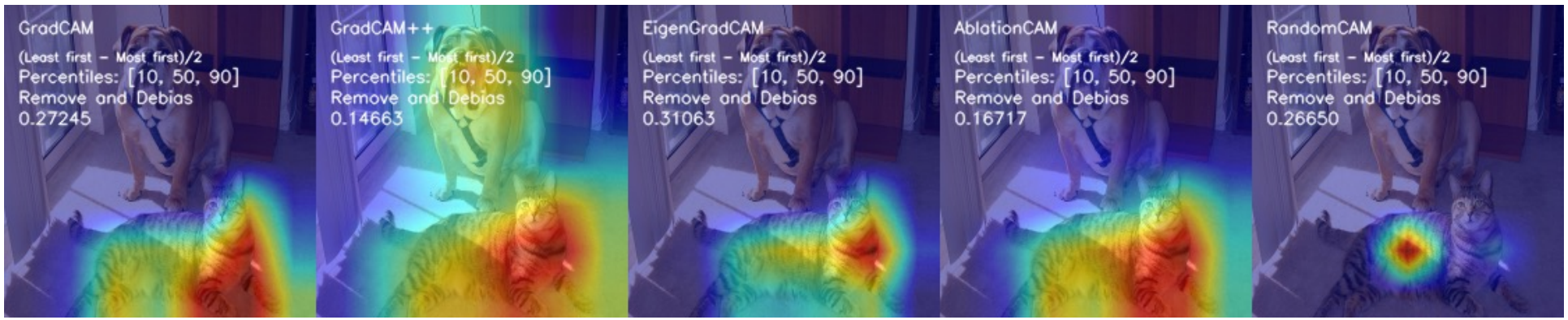
✓ 可视化层激活、可视化层权重、检索最大程度激活神经元的图像、使用 t-SNE 嵌入隐藏层神经元、遮挡图像的部分等



可视化 AlexNet 的 conv1 权重


Layer 1
True Label: Pomeranian
Pomeranian
Tennis ball
Keeshond
Pekinese
True Label: Car Wheel
Car wheel
Racer
Cab
Police van
True Label: Afghan Hound
Afghan hound
Gordon setter
Irish setter
Mortarboard
Fur coat
Academic gown
Australian terrier
Ice lolly
Vizsla
Neck brace
Layer 4
Layer 5

GradCAM



https://github.com/jacobgil/pytorch-grad-cam

大纲

- 背景引入
- CNN基础知识
- 深度学习技巧
- 深度学习工具箱
- 经典网络架构
- Transformer
- Mamba

深度学习工具箱

❑ 深度学习工具箱

- **PyTorch:** tensors and dynamic neural networks in python (Python);
- **TensorFlow** : Using data flow graphs (C++, Python);
- **MatConvNet:** A matlab toolbox implementing CNNs (Matlab);
- **Caffe** : C++, Python;
- **Keras:** A theano based deep learning library;
- **Torch:** provides a Matlab-like environment for ML algorithms in lua (Lua);
- **Theano:** CPU/GPU symbolic expression compiler in python (Python);
- **MXNet:** mix symbolic and imperative programming (Python, R, Julia…);
- **CNTK :** a unified deep-learning toolkit by Microsoft (C++, Python).
- **PaddlePaddle**: An Open-Source Deep Learning Platform (百度)
- **MindSpore**: 面向“端-边-云”全场景设计的AI框架（华为）
- **MegEngine**： 一个快速、可拓展、易于使用且支持自动求导的深度学习框架（旷视）
- **Jittor**：一个基于即时编译和元算子的高性能深度学习框架（清华大学）

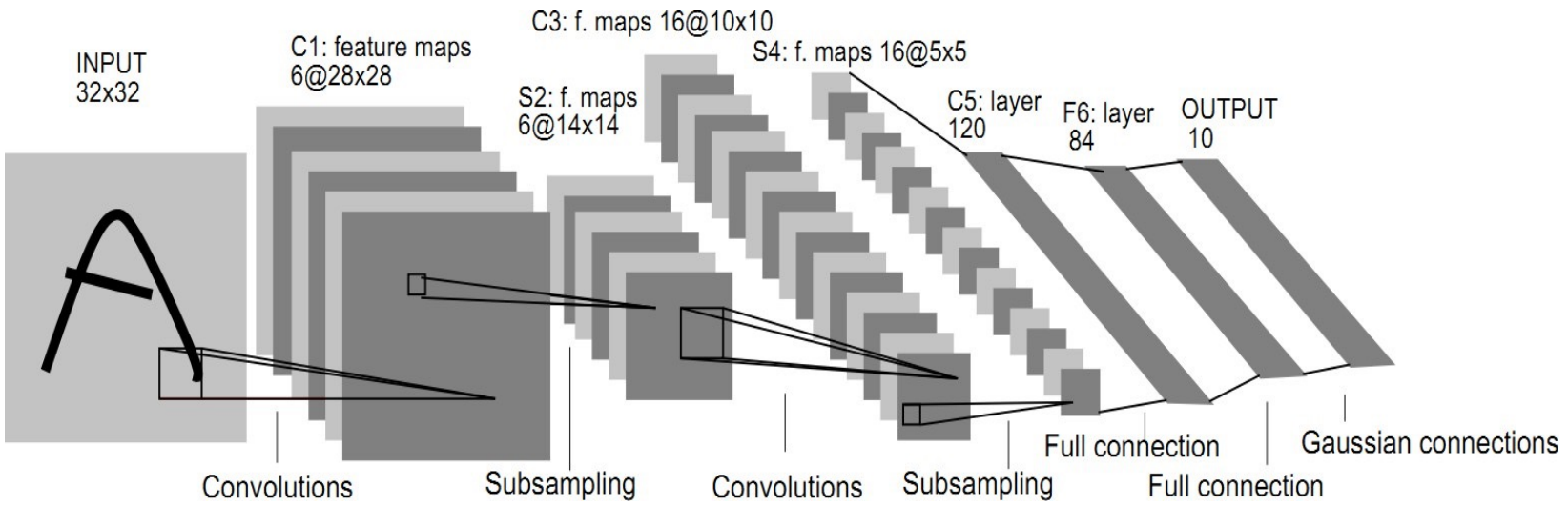
大纲

- ❑ 背景引入
- ❑ CNN基础知识
- ❑ 深度学习技巧
- ❑ 深度学习工具箱
- ❑ 经典网络架构
- ❑ Transformer
- ❑ Mamba

经典网络架构

❑ LeNet

- 卷积（1.局部连接 2.空间权重共享 3.权重共享是深度学习中的关键）
- 全连接输出
- 通过BackPropagation训练
- 所有这些仍然是现代卷积神经网络的基本组成部分！



"Gradient-based learning applied to document recognition", LeCun et al. 1998
"Backpropagation applied to handwritten zip code recognition", LeCun et al. 1989

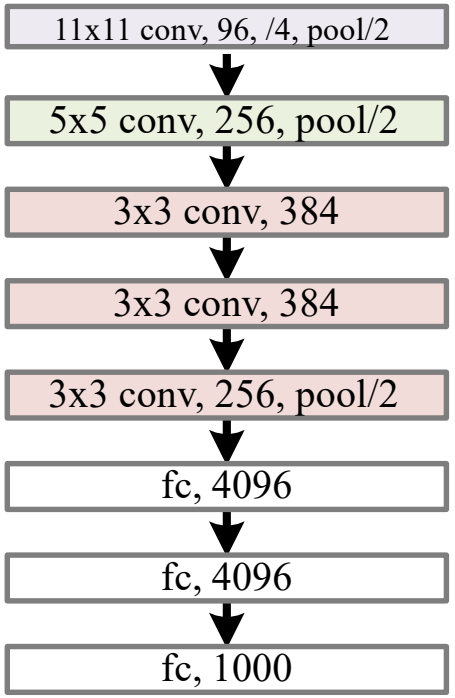
A Demo from 1993 of 32-year-old Yann LeCun showing off the World's first Convolutional Network for Text Recognition

"Gradient-based learning applied to document recognition", LeCun et al. 1998
"Backpropagation applied to handwritten zip code recognition", LeCun et al. 1989

经典网络架构

❑ AlexNet

- **LeNet风格的骨干网络**
  - **Five convolutional layers**
  - **60 million** parameters, 650000 neurons
  - ImageNet-1K top-1 error rate: **37.5%**
- **ReLU 非线性激活函数**
  - 采用了ReLU [Nair and Hinton, ICML 2010]
  - 加速训练，比tanh激活函数更好
- **Dropout [Hinton et al 2012]**
  - 网络内集成
  - 减少过拟合
- **数据增强**
  - 标签保留变换
  - 减少过度拟合



"ImageNet Classification with Deep Convolutional Neural Networks", Krizhevsky, Sutskever, Hinton. NIPS 2012

经典网络架构

- AlexNet

ImageNet Classification with Deep Convolutional Neural Networks

Alex Krizhevsky
University of Toronto
kriz@cs.utoronto.ca

Ilya Sutskever
University of Toronto
ilya@cs.utoronto.ca

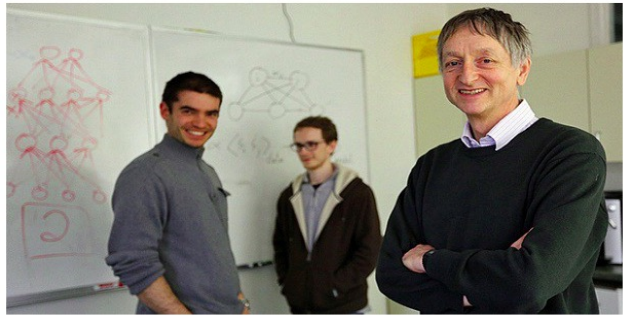
Geoffrey E. Hinton
University of Toronto
hinton@cs.utoronto.ca

Alex Krizhevsky

University of Toronto

Verified email at cs.toronto.edu

Machine Learning



GET MY OWN PROFILE

TITLE	CITED BY	YEAR
Imagenet classification with deep convolutional neural networks A Krizhevsky, I Sutskever, GE Hinton Advances in neural information processing systems 25	144864 *	2012

Cited by	VIEW ALL	
	All	Since 2018
Citations	243119	204137
h-index	32	31
i10-index	37	37

40000

经典网络架构

- AlexNet

ImageNet Classification with Deep Convolutional Neural Networks

Alex Krizhevsky
University of Toronto
kriz@cs.utoronto.ca

Ilya Sutskever
University of Toronto
ilya@cs.utoronto.ca

Geoffrey E. Hinton
University of Toronto
hinton@cs.utoronto.ca

Ilya Sutskever

FOLLOW

Co-Founder and Chief Scientist of OpenAI
Verified email at openai.com - Homepage

Machine Learning Neural Networks Artificial Intelligence Deep Learning

GET MY OWN PROFILE

TITLE	CITED BY	YEAR
Imagenet classification with deep convolutional neural networks A Krizhevsky, I Sutskever, GE Hinton Advances in neural information processing systems 25	145604 *	2012

Cited by VIEW ALL

	All	Since 2018
Citations	449692	389467
h-index	82	80
i10-index	112	109

76000

经典网络架构

- AlexNet

ImageNet Classification with Deep Convolutional Neural Networks

Alex Krizhevsky
University of Toronto
kriz@cs.utoronto.ca

Ilya Sutskever
University of Toronto
ilya@cs.utoronto.ca

Geoffrey E. Hinton
University of Toronto
hinton@cs.utoronto.ca

Geoffrey Hinton

关注

Emeritus Prof. Computer Science, University of Toronto
在 cs.toronto.edu 的电子邮件经过验证 - 首页

machine learning psychology artificial intelligence cognitive science computer science

创建我的个人资料

标题	引用次数	年份
Imagenet classification with deep convolutional neural networks A Krizhevsky, I Sutskever, GE Hinton Advances in neural information processing systems 25	145027 *	2012

引用次数	查看全部	
	总计	2018 年至今
引用	728043	521626
h 指数	180	133
i10 指数	445	348

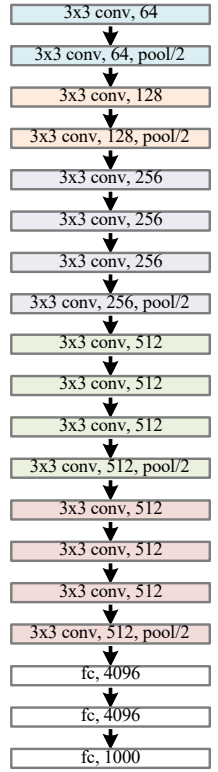
103000

经典网络架构

❑ VGG-16/19

“16 layers are beyond my imagination!”
-- after ILSVRC 2014 result was announced.

- ✓ **简单来说，就是“加深“**
- ✓ **模块化设计**
  - 3x3 Conv as the module
  - Stack the same module
  - Same computation for each module
- ✓ **阶段性训练**
  - VGG-11 => VGG-13 => VGG-16
  - 我们需要更好的初始化.....

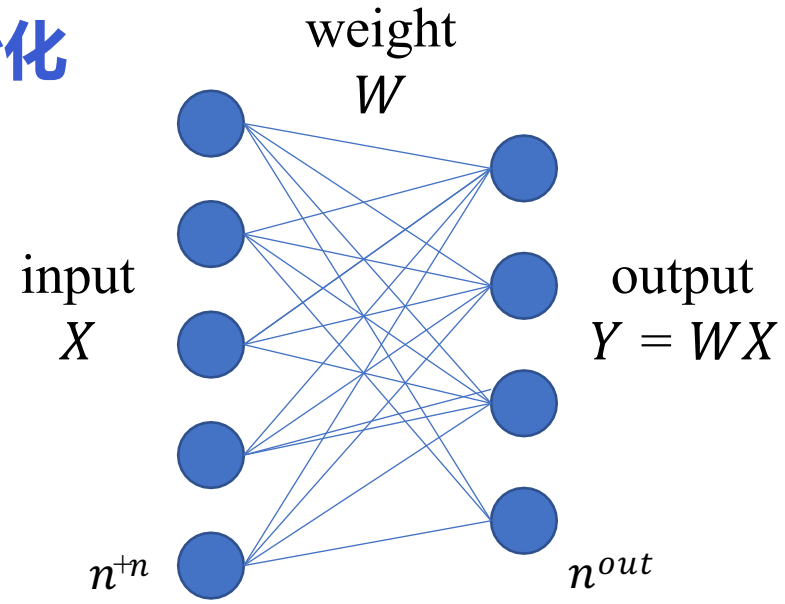


“Very Deep Convolutional Networks for Large-Scale Image Recognition”, Simonyan & Zisserman. ICLR 2015.

经典网络架构

❑ VGG-16/19

✓ 初始化



If:

- Linear activation
- x, y, w: independent

Then:

1-layer:

$$Var[y] = (n^{in} Var[w]) Var[x]$$

Multi-layer:

$$Var[y] = (\prod_{d} n_d^{in} Var[w_d]) Var[x]$$

LeCun et al 1998 “Efficient Backprop”
Glorot & Bengio 2010 “Understanding the difficulty of training deep feedforward neural networks”

经典网络架构

❑ VGG-16/19

✓ 初始化

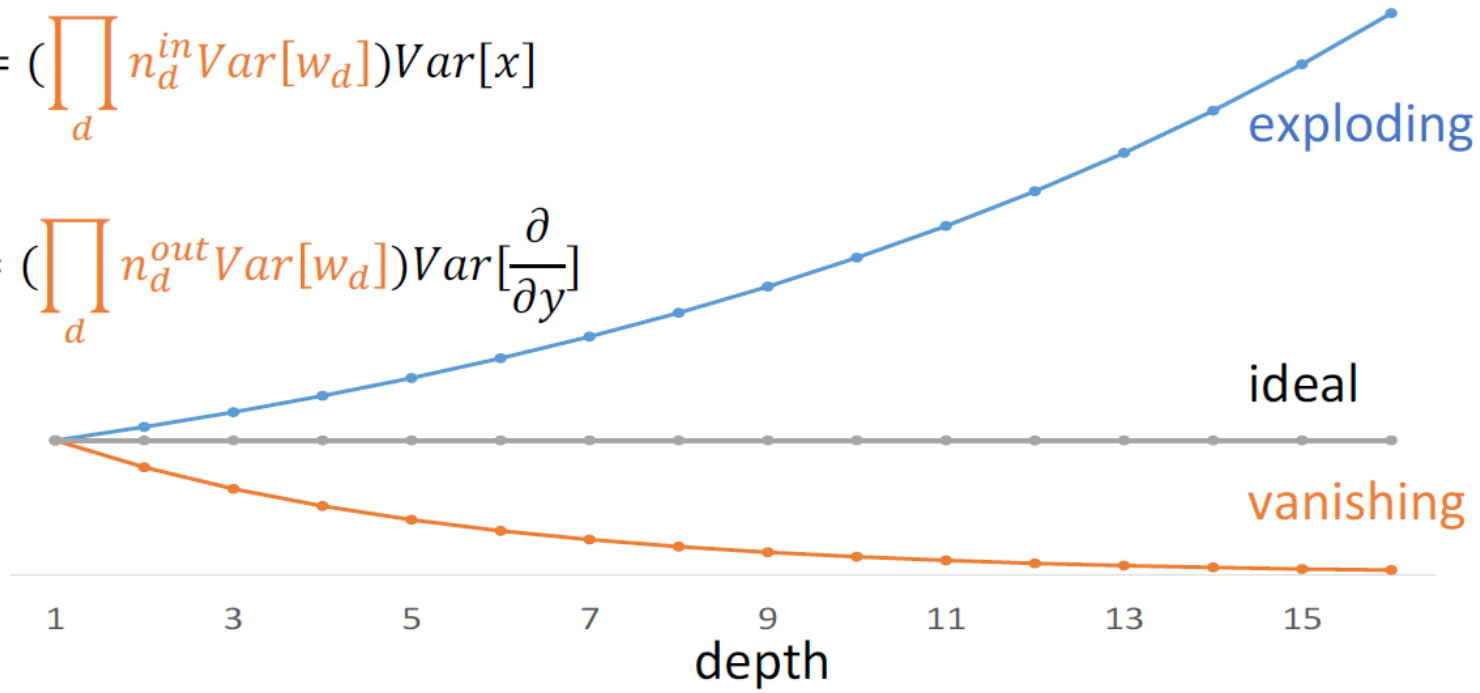
前向（响应）和后向（梯度）信号都可能消失/爆炸

Forward:

$$Var[y] = (\prod_d n_d^{in} Var[w_d])Var[x]$$

Backward:

$$Var\left[\frac{\partial}{\partial x}\right] = (\prod_d n_d^{out} Var[w_d])Var[\frac{\partial}{\partial y}]$$



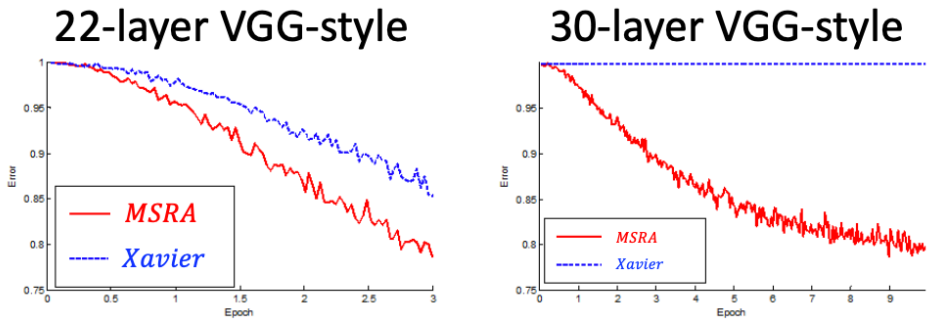
LeCun et al 1998 “Efficient Backprop”
Glorot & Bengio 2010 “Understanding the difficulty of training deep feedforward neural networks”

经典网络架构

❑ VGG-16/19

✓ 初始化

Xavier/MSRA初始化



*Figures show the beginning of training

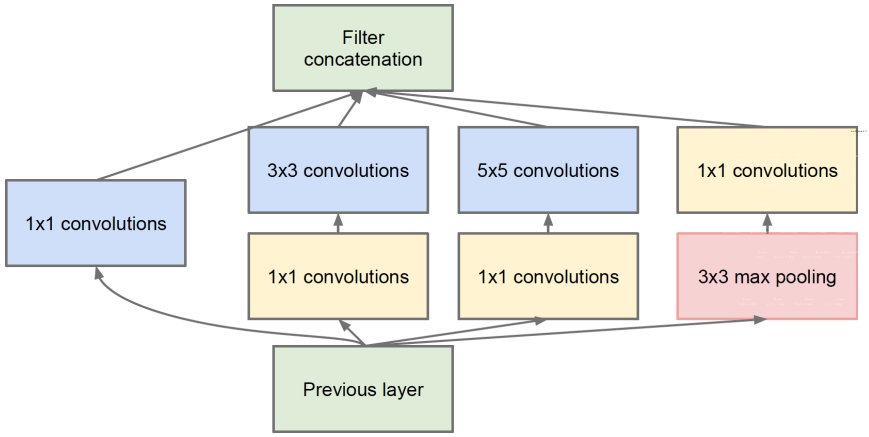
- 从头开始训练 VGG-16/19所需
- 可以使用 MSRA初始化来训练更深的 (>20层) VGG 式网络
  - 但更深的网络并不意味着更好
- 推荐用于微调中新初始化的层

Kaiming He, Xiangyu Zhang, Shaoqing Ren, & Jian Sun. “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification”. ICCV 2015.

经典网络架构

❑ GoogleNet/Inception

- ✓ 精度上有小的提升
- ✓ GoogleNet核心模块
  - Multiple branches
    - e.g., 1x1, 3x3, 5x5, pool
  - Shortcuts
    - stand-alone 1x1, merged by concat.
  - Bottleneck
    - Reduce dim by 1x1 before expensive 3x3/5x5 conv



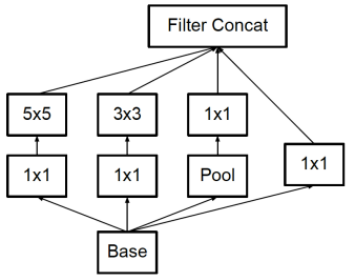
Szegedy et al. “Going deeper with convolutions”. arXiv 2014 (CVPR 2015).

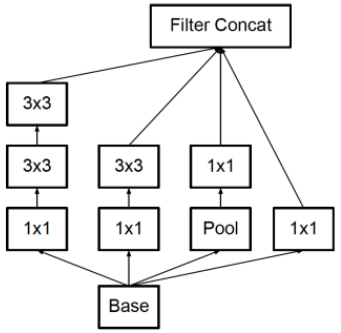
经典网络架构

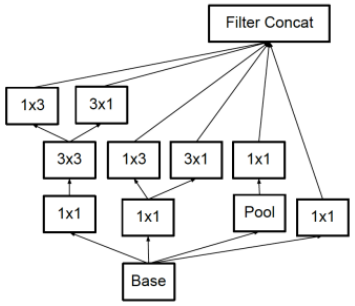
❑ GoogleNet/Inception

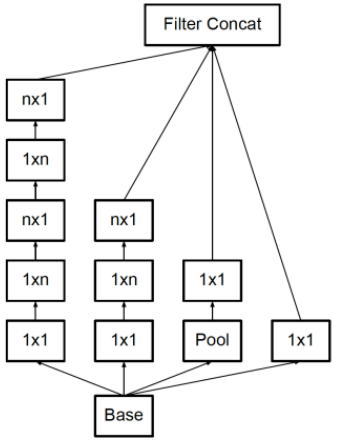
✓ 更多模板，但保留了相同的三个主要属性

- Multiple branches
- Shortcuts (1x1, concate.)
- Bottleneck







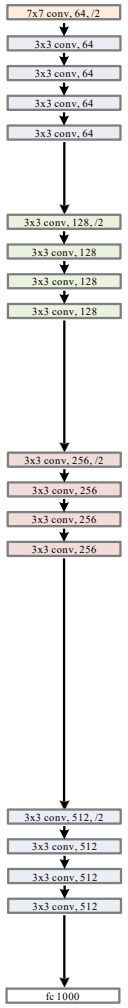


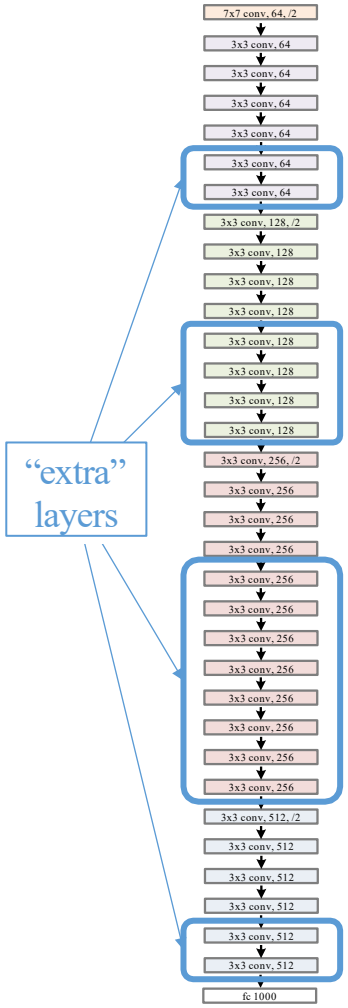
Szegedy et al. “Rethinking the Inception Architecture for Computer Vision”. arXiv 2015 (CVPR 2016).

经典网络架构

❑ ResNets

浅层模型
(18 layers)





深层模型
(34 layers)

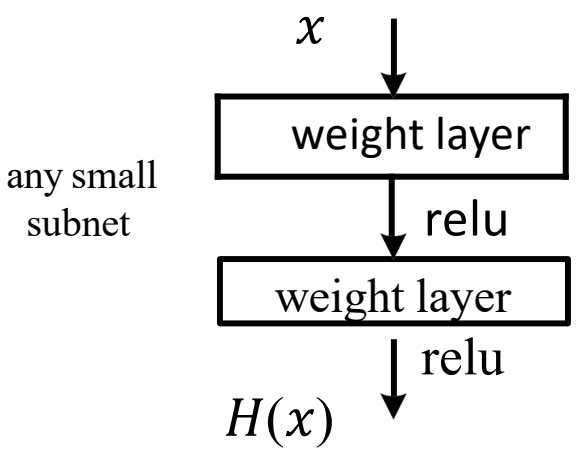
- 更丰富的解空间
- 更深的模型不应该有更高的训练误差
- 解决方案:
  - 原始层：从学习的较浅模型复制
  - 额外层：设置为identity
  - 至少有相同的训练误差
- 优化困难：求解器在深入时无法找到解决方案……

Kaiming He, Xiangyu Zhang, Shaoqing Ren, & Jian Sun. "Deep Residual Learning for Image Recognition". CVPR 2016.

经典网络架构

❑ ResNets

✓ plain net



$H(x)$ is any desired mapping,

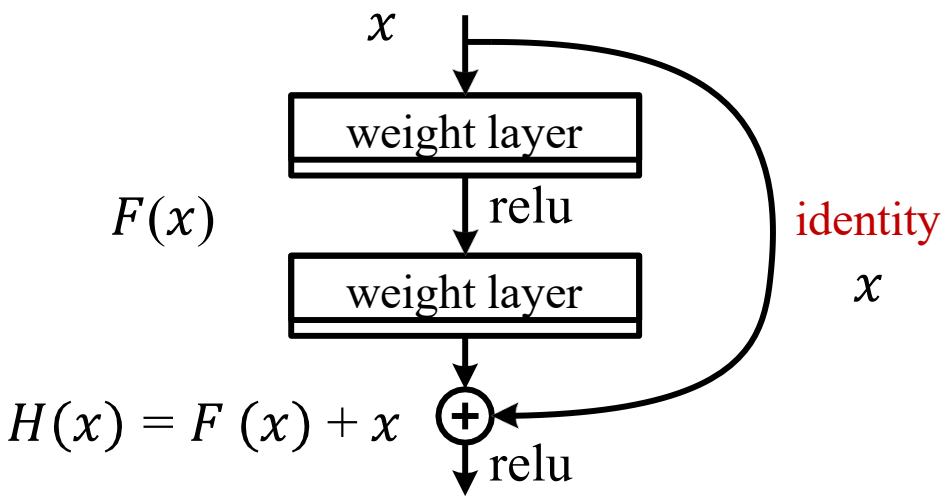
hope the small subnet fit $H(x)$

Kaiming He, Xiangyu Zhang, Shaoqing Ren, & Jian Sun. “Deep Residual Learning for Image Recognition”. CVPR 2016.

经典网络架构

❑ ResNets

✓ Residual net



$H(x)$ is any desired mapping,

~~hope the small subnet fit $H(x)$~~

hope the small subnet fit $F(x)$

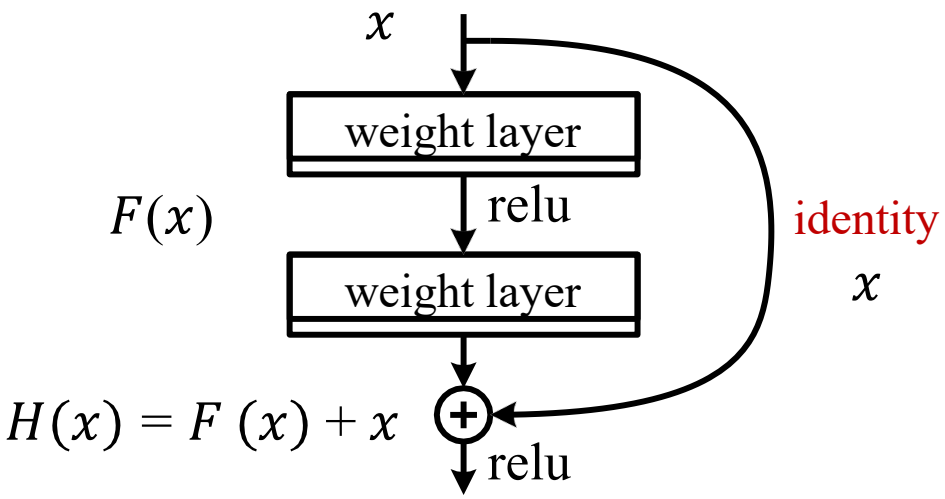
let $H(x) = F(x) + x$

Kaiming He, Xiangyu Zhang, Shaoqing Ren, & Jian Sun. "Deep Residual Learning for Image Recognition". CVPR 2016.

经典网络架构

❑ ResNets

- $F(x)$ is a residual mapping w.r.t. identity

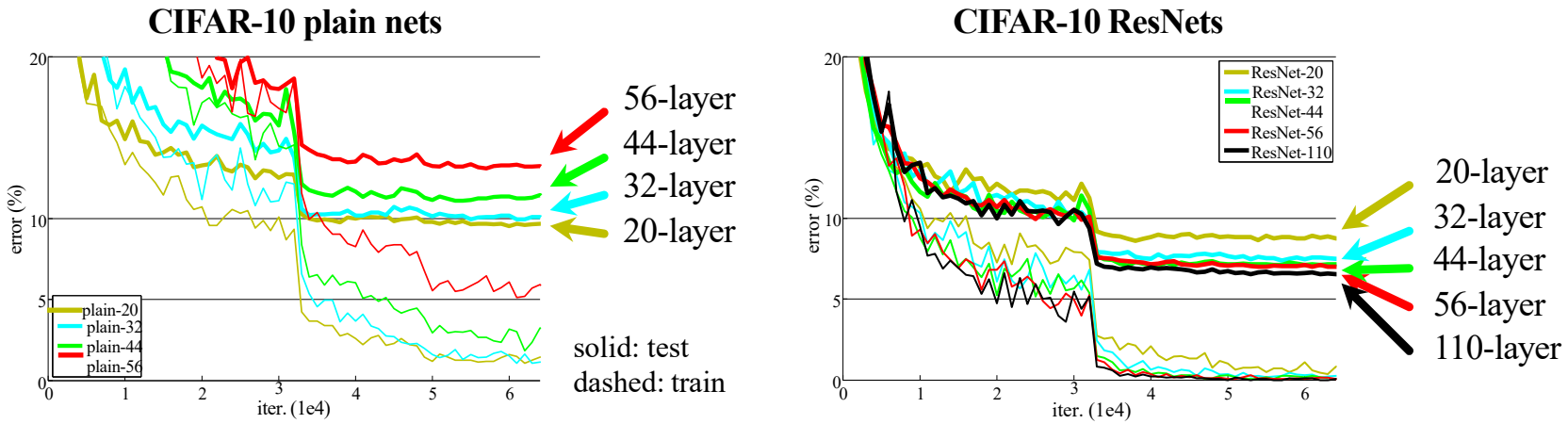


- If identity were optimal, easy to set weights as 0
- If optimal mapping is closer to identity, easier to find small fluctuations

Kaiming He, Xiangyu Zhang, Shaoqing Ren, & Jian Sun. “Deep Residual Learning for Image Recognition”. CVPR 2016.

经典网络架构

❑ CIFAR-10实验

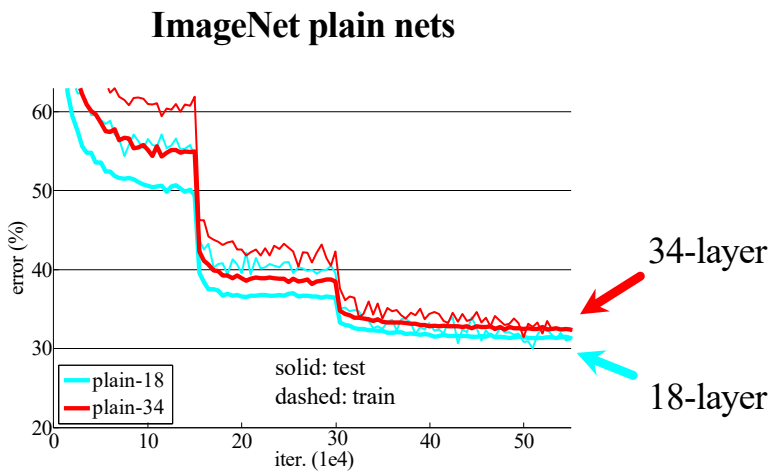


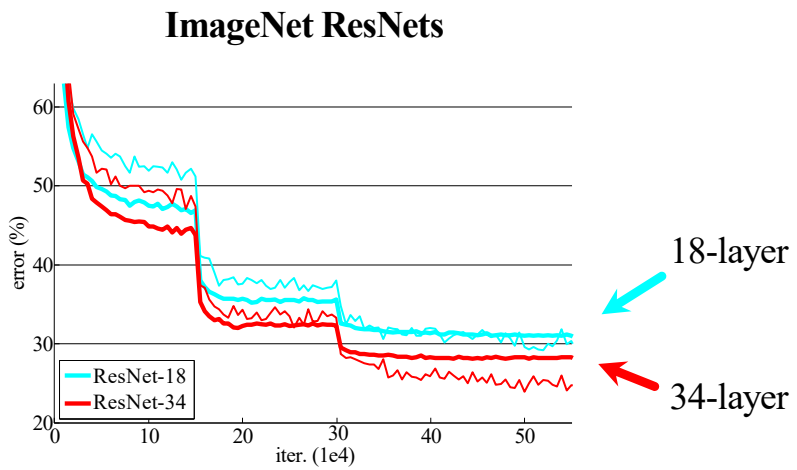
- 深度 ResNet 的训练没有困难
- 更深的 ResNet 具有更低的训练误差和更低的测试误差

Kaiming He, Xiangyu Zhang, Shaoqing Ren, & Jian Sun. “Deep Residual Learning for Image Recognition”. CVPR 2016.

经典网络架构

❑ ImageNet实验





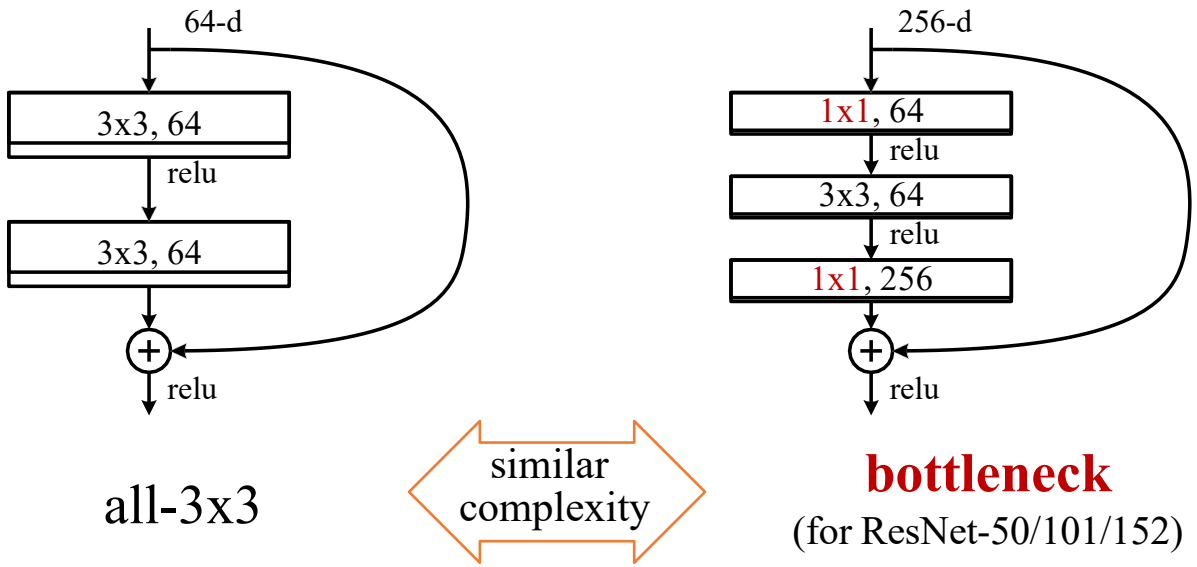
- 深度 ResNet 的训练没有困难
- 更深的 ResNet 具有更低的训练误差和更低的测试误差

Kaiming He, Xiangyu Zhang, Shaoqing Ren, & Jian Sun. “Deep Residual Learning for Image Recognition”. CVPR 2016.

经典网络架构

❑ ImageNet实验

✓ 更深网络的实用设计

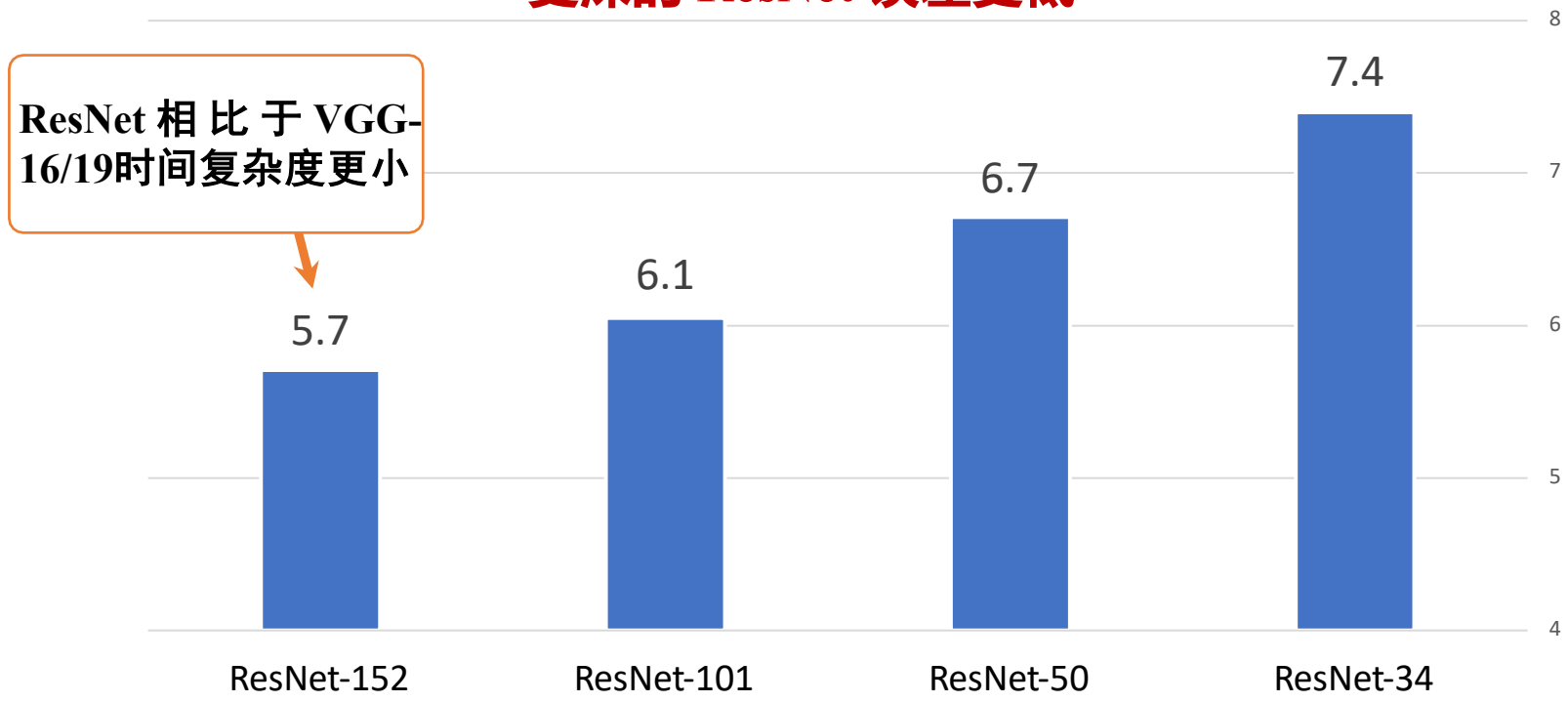


Kaiming He, Xiangyu Zhang, Shaoqing Ren, & Jian Sun. “Deep Residual Learning for Image Recognition”. CVPR 2016.

经典网络架构

❑ ImageNet实验

- **更深的 ResNet 误差更低**



10-crop testing, top-5 val error (%)

Kaiming He, Xiangyu Zhang, Shaoqing Ren, & Jian Sun. "Deep Residual Learning for Image Recognition". CVPR 2016.

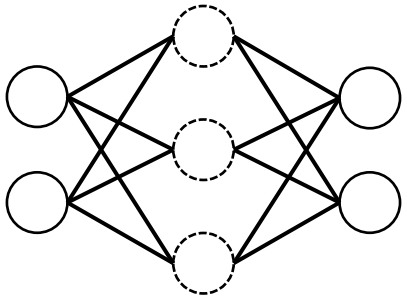
大纲

- 背景引入
- CNN基础知识
- 深度学习技巧
- 深度学习工具箱
- 经典网络架构
- Transformer
- Mamba

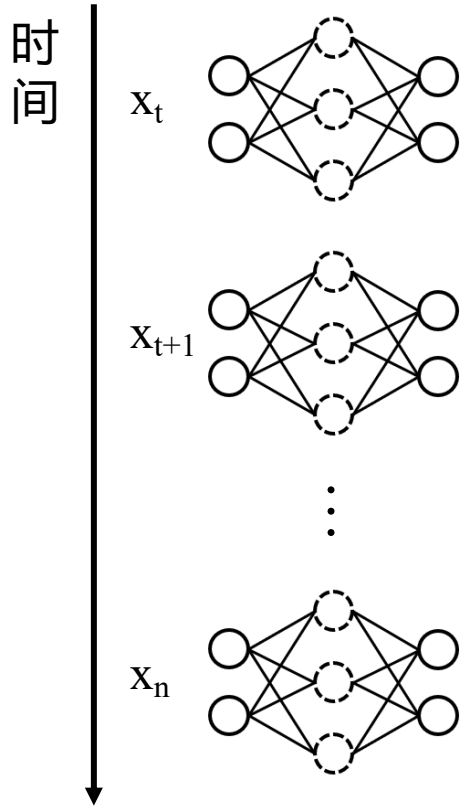
Transformer

❑ RNN

✓ **参考读物:A Critical Review of Recurrent Neural Networks for Sequence Learning（ArXiv 2015）**



输入　隐藏层　输出



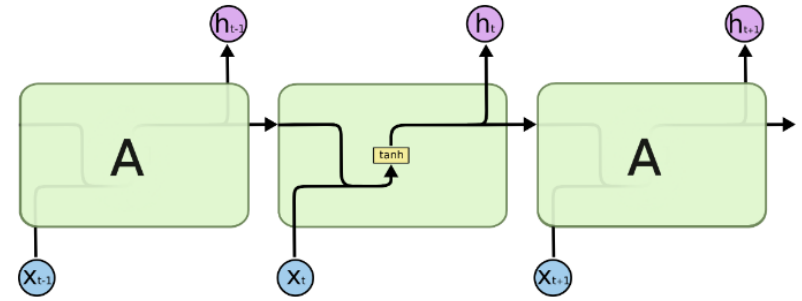
- 层级拓展并非神经元数量的增加，而是表示隐层在不同时刻的状态
- $s_i = f(\sum_n^N (w_{in}^i \cdot x_n^i + b_i))$
- 矩阵形式：

$$S = f(W_{in}X + b)$$

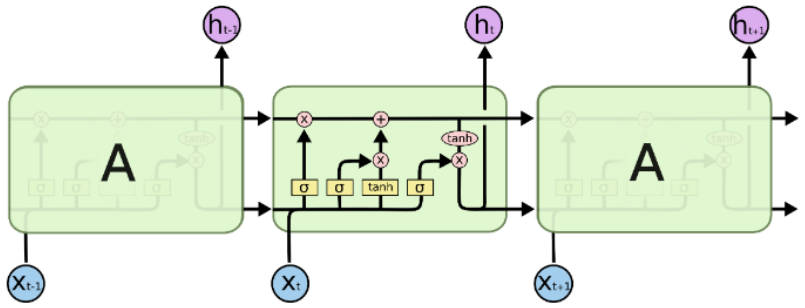
Transformer

❑ LSTM

✓ **参考读物: Understanding LSTM Networks（2015 Blog）**



RNN



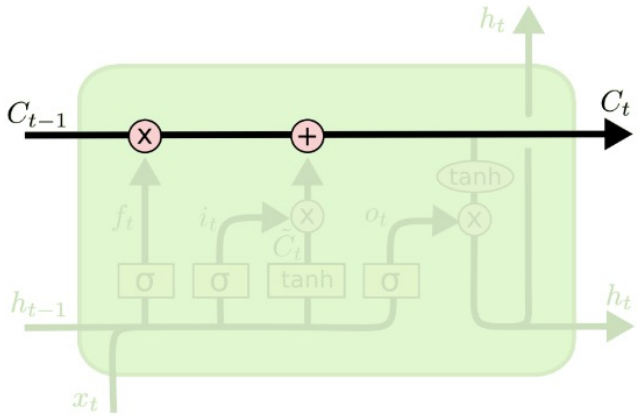
LSTM

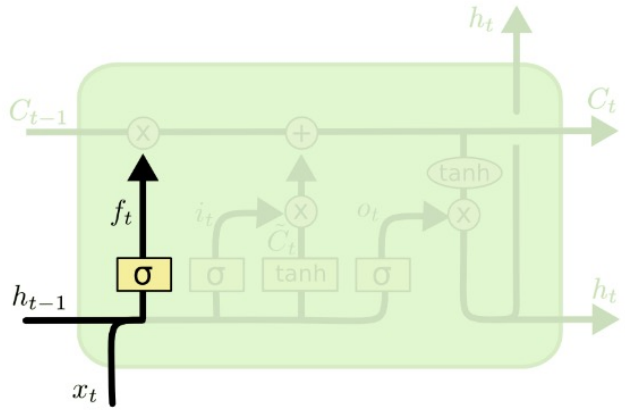
□ LSTM解决了RNN随着信息量的提高，会出现梯度消失，梯度爆炸，长距离依赖性差等问题

Transformer

❑ LSTM

✓ LSTM结构分解





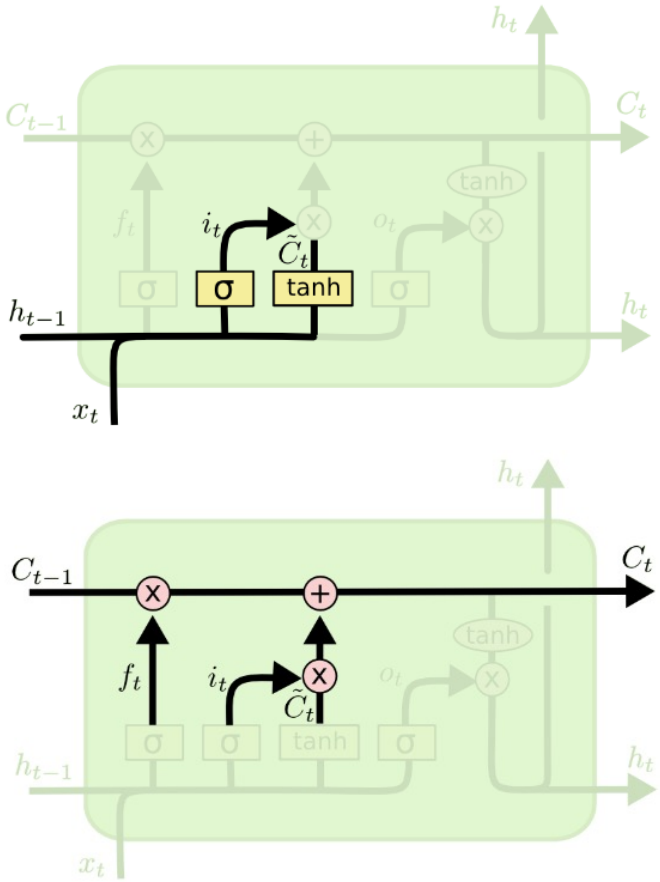
- 细胞状态（Cell State）：细胞状态类似于传送带。直接在整个链上运行，只有一些少量的线性交互。

- 遗忘门：决定从细胞状态中丢弃什么信息。遗忘门会读取上一个输出和当前输入，做Sigmoid 非线性映射，然后将输出与细胞状态相乘。

Transformer

❑ LSTM

✓ LSTM结构分解

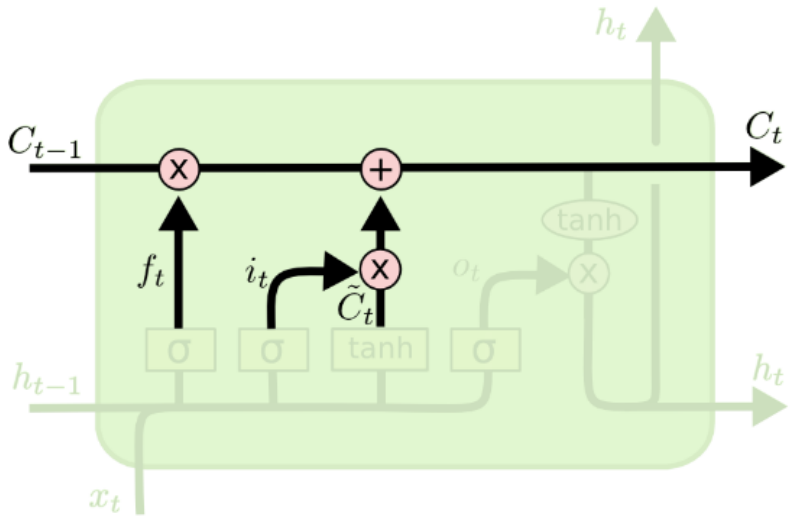


- 输入门：确定什么样的新信息被存放在细胞状态中。其中Sigmoid层决定什么值将要更新，tanh层创建一个新的候选值向量，加入到状态中 。
- 遗忘门：决定从细胞状态中丢弃什么信息。遗忘门会读取上一个输出和当前输入，做Sigmoid 非线性映射，然后将输出与细胞状态相乘。

Transformer

❑ LSTM

✓ LSTM结构分解



❑ 更新细胞状态

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

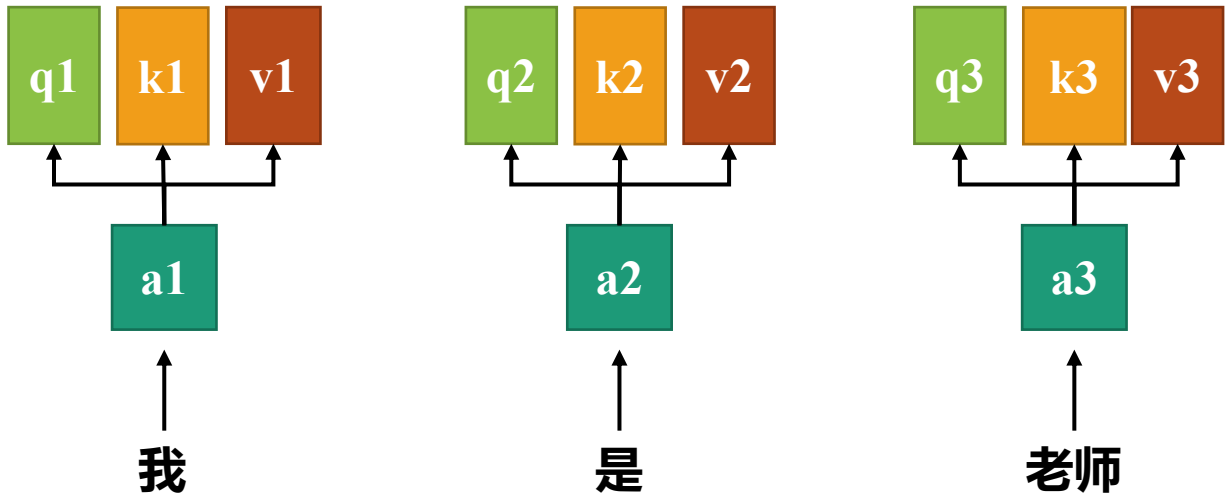
把旧状态C_{t-1}与f_t相乘，丢弃掉遗忘门确定需要丢弃的信息。接着加上$i_t * \tilde{C}_t$这就是新的候选值。

Transformer

❑ Self-Attention

✓ Attention is All you Need（NeurIPS 2017）

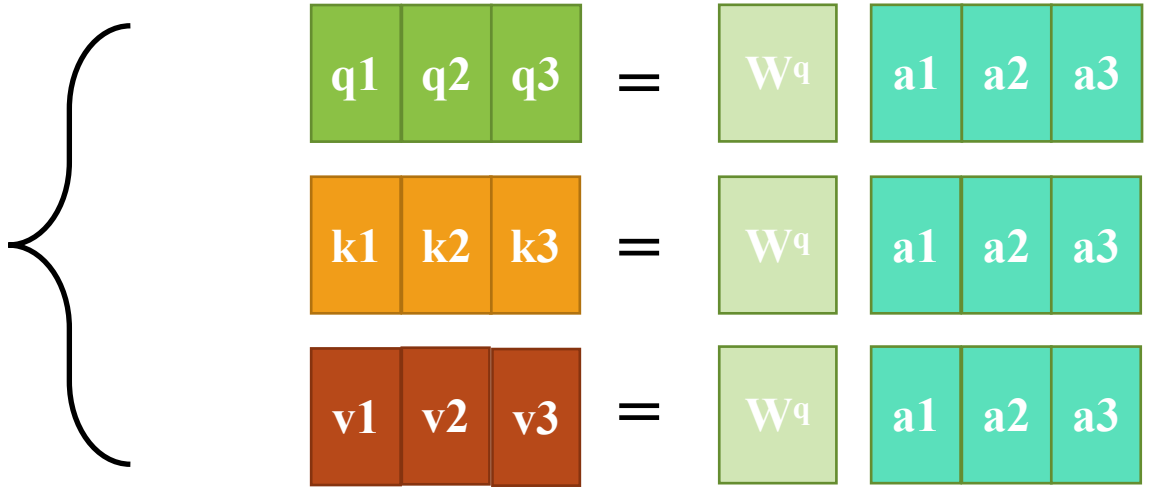
- 不同长度的序列经过词嵌入形成相同长度的token，每个token又形成Q，K，V三个向量

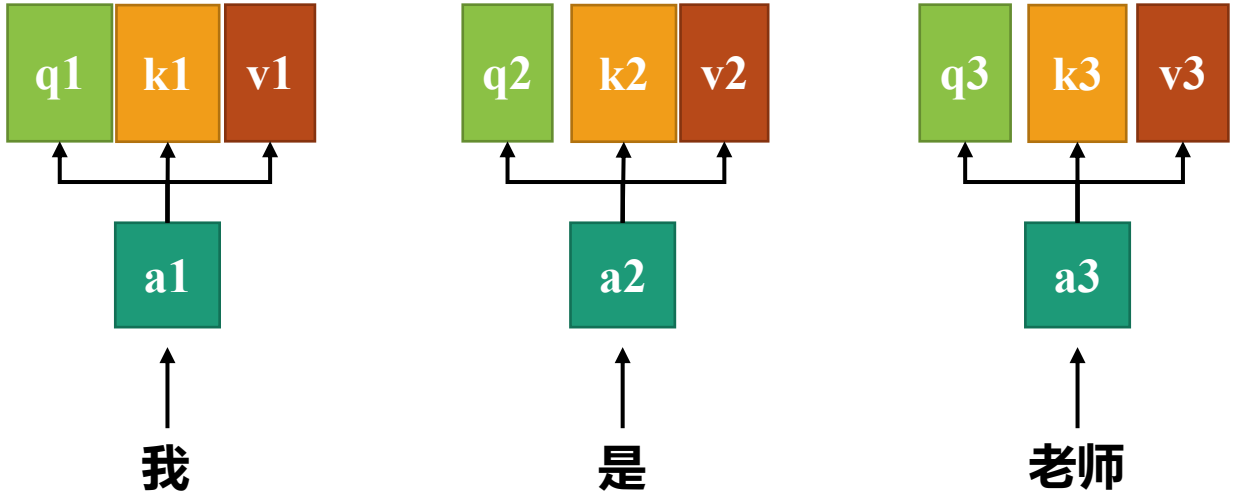


Transformer

❑ Self-Attention

$$q^i = W^q a^i$$
$$k^i = W^k a^i$$
$$v^i = W^v a^i$$





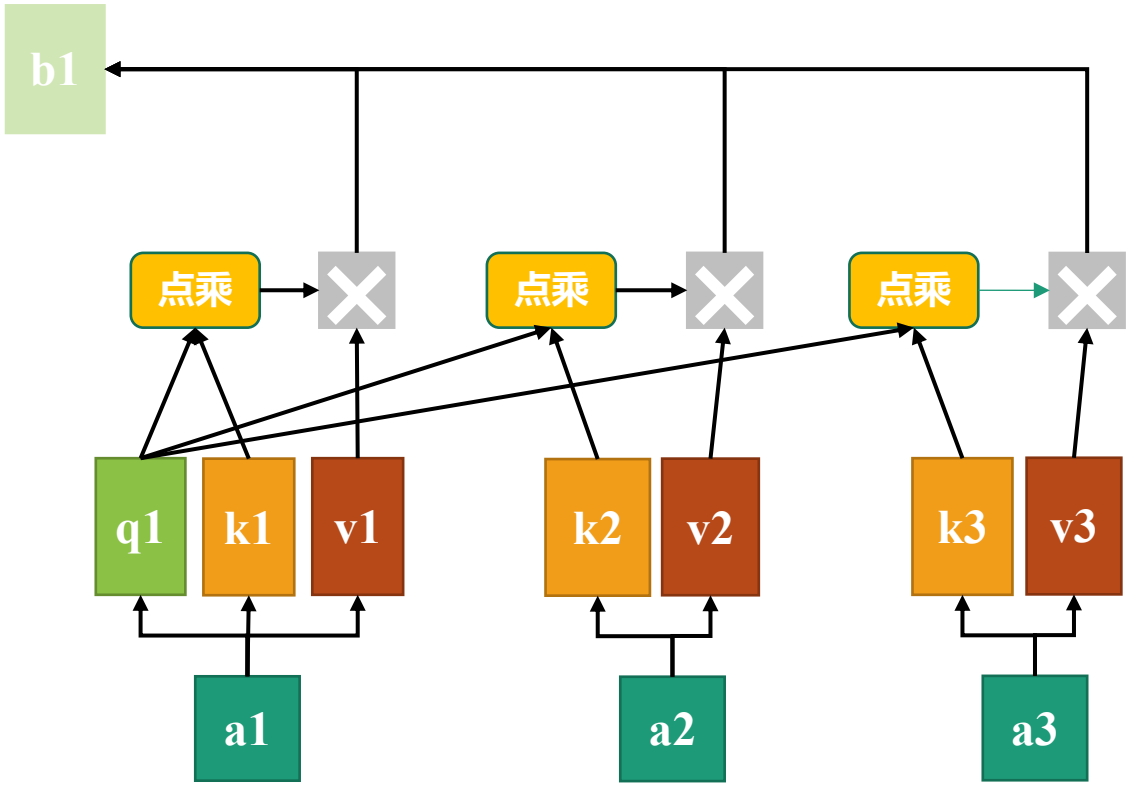
Transformer

❑ Self-Attention

✓ 每个Query分别对每个Key做Attention

$$b_{1,i} = q^1 \cdot k^i / \sqrt{d}$$

d为Query和Key的维度数，
起标准化作用

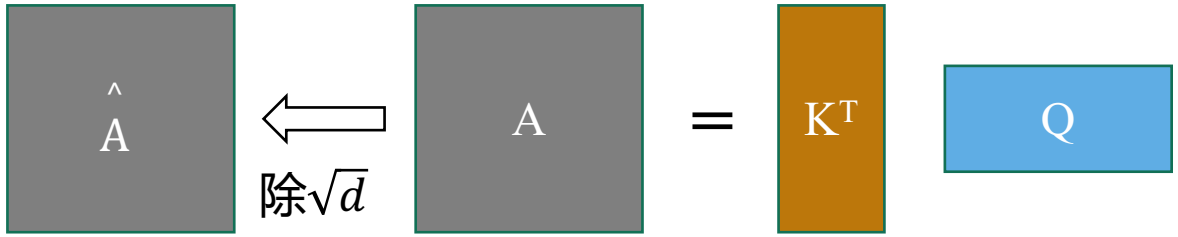


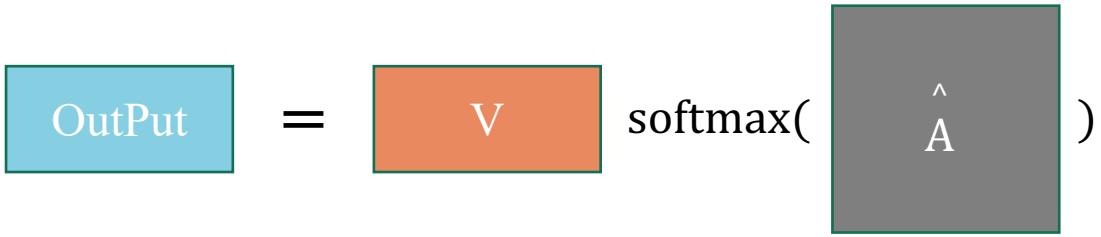
Transformer

❑ Self-Attention

✓ 将得到的Attention Map与Value矩阵相乘

$$\text{out} = \text{softmax}(\frac{QK^T}{\sqrt{d_k}}) * V$$

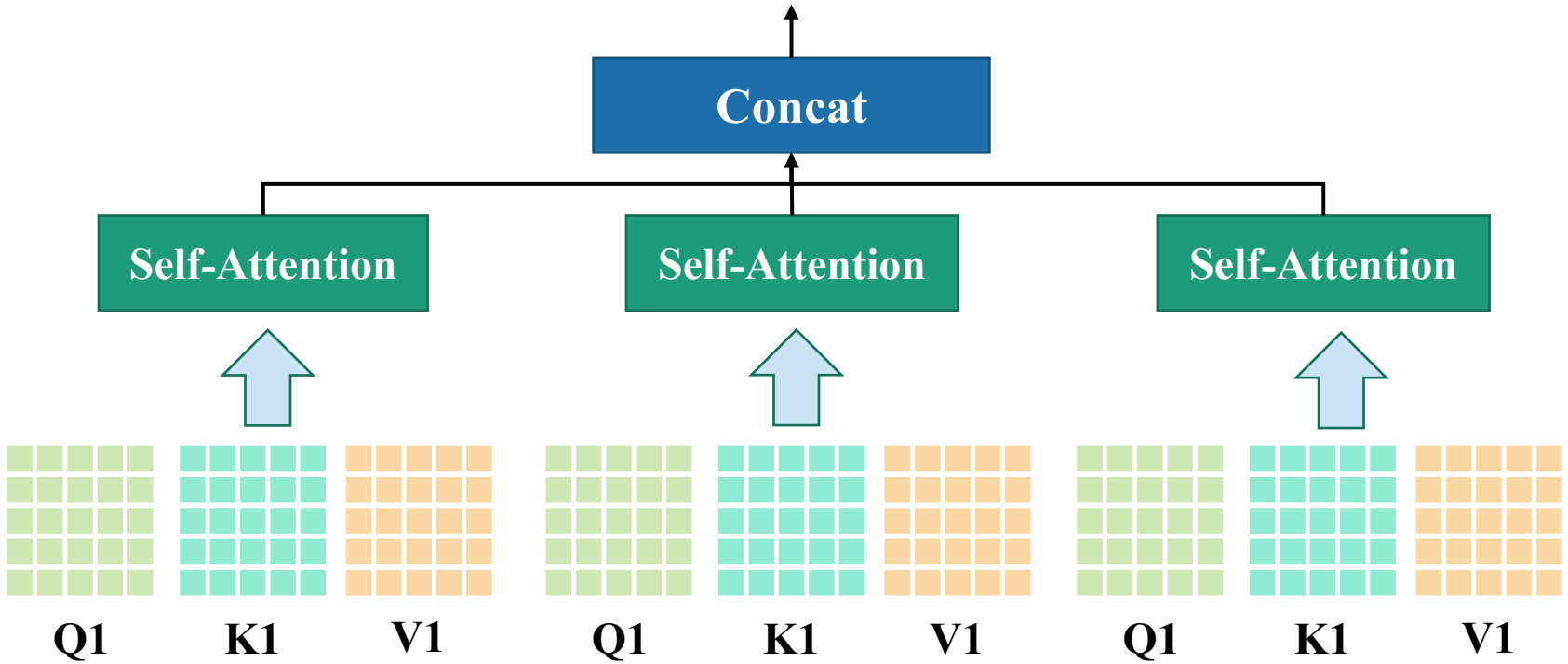




Transformer

❑ Self-Attention

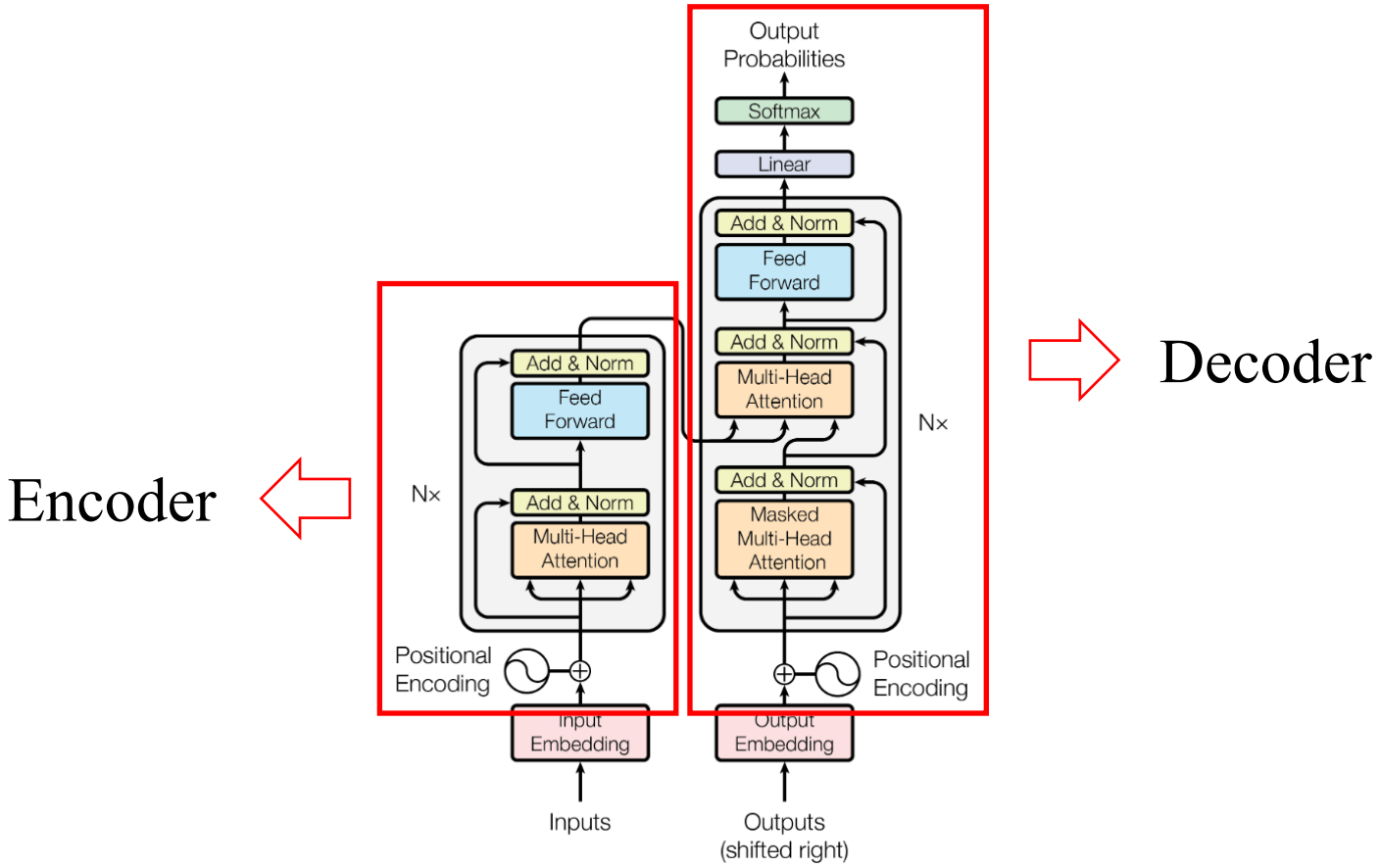
✓ Multi-Head Attention通过把输入降维后，经过多通道即多个Head分别进行attention，再将各个输出拼接成原始的维度



Transformer

❑ Transformer

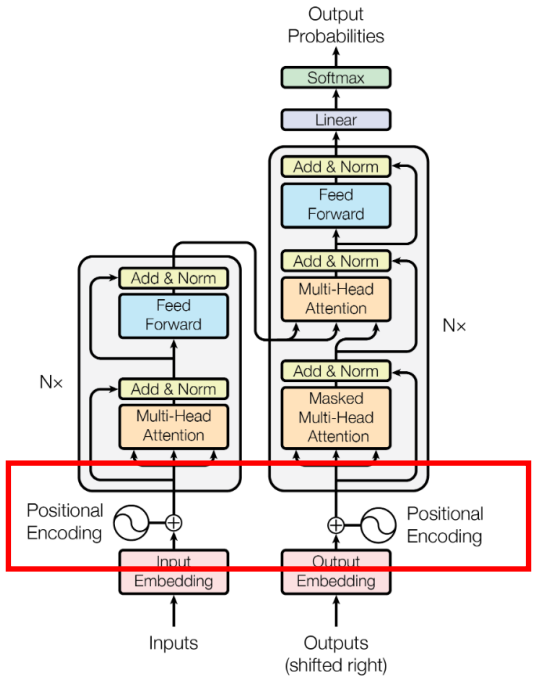
✓ Attention is All you Need（NeurIPS 2017）

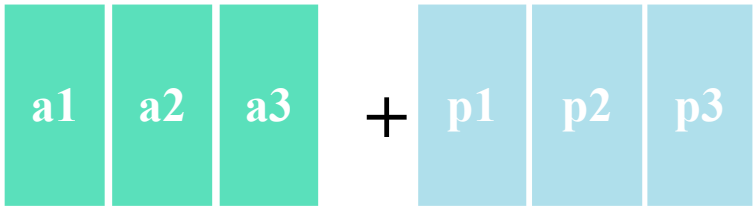


Transformer

❑ Position Embedding

✓ **由于Transformer模型不包含递归和卷积，为了使模型能够利用序列的顺序，必须注入一些关于token在序列中的相对或绝对位置的信息**

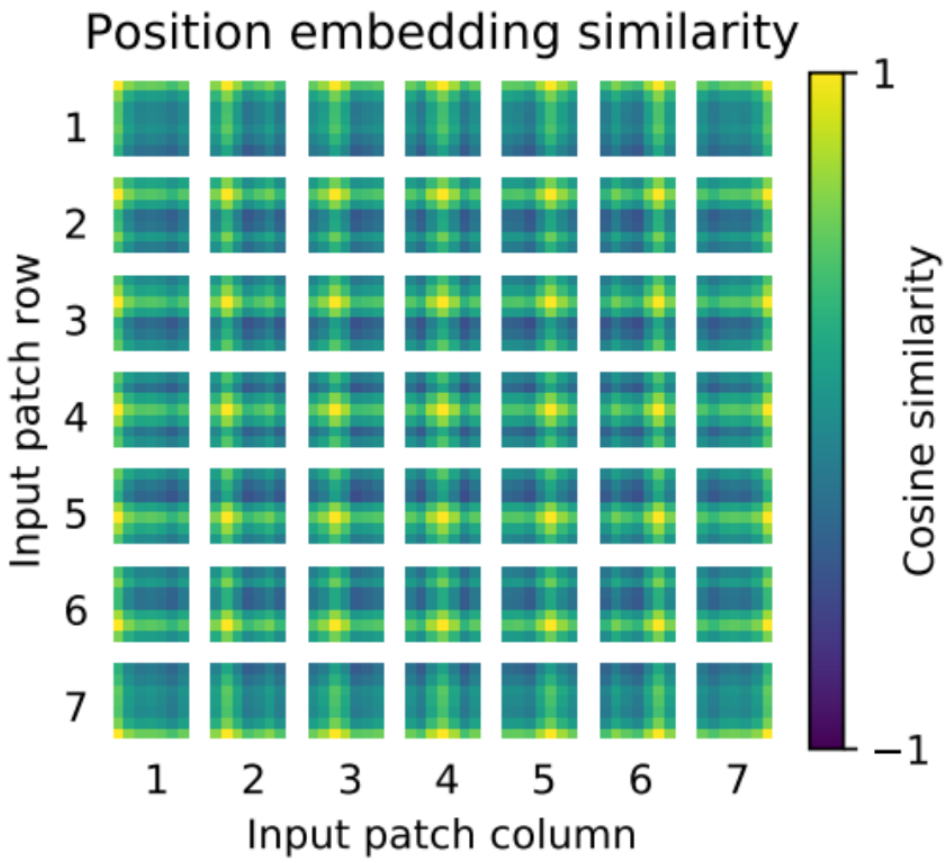




❑ Position embedding与对应的token的具有相同的维度，所以在处理时可以相加

Transformer

❑ Position Embedding

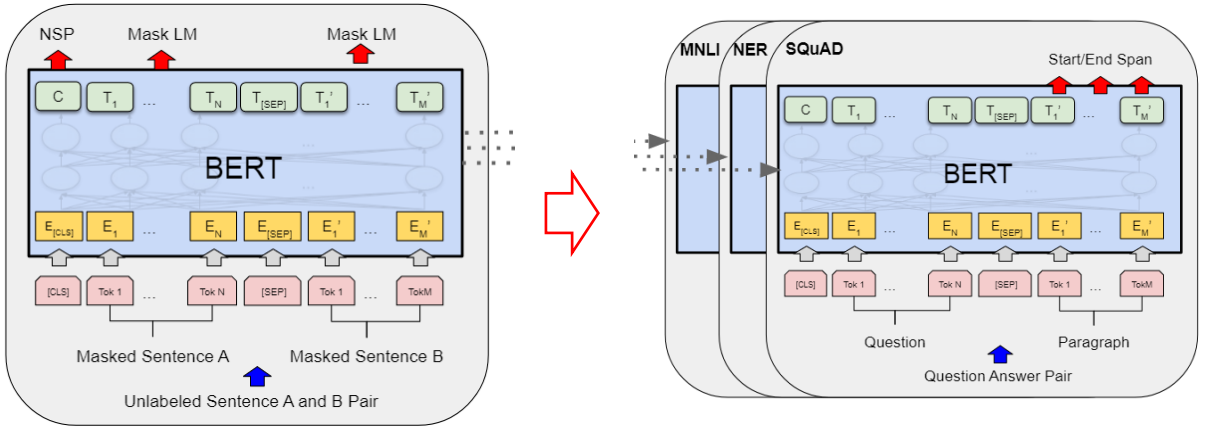


- Position embedding的相似性分析，位置越接接近，patches之间的相似度越高；相同行/列的patches有相似的embeddings

Transformer

❑ BERT

✓ **BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding （NAACL 2019）**



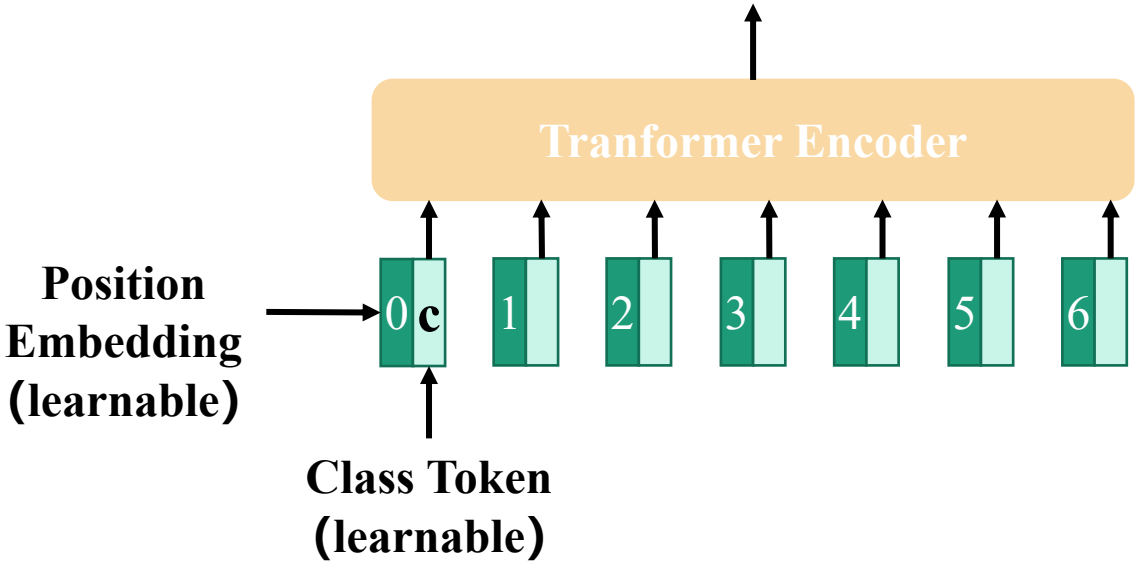
预训练阶段　　微调阶段

- BERT发表时在11个NLP任务上都取得了SOTA的效果！

[BERT的中文预训练模型 GitHub - ymcui/Chinese-BERT-wwm]

Transformer

❑ Class token

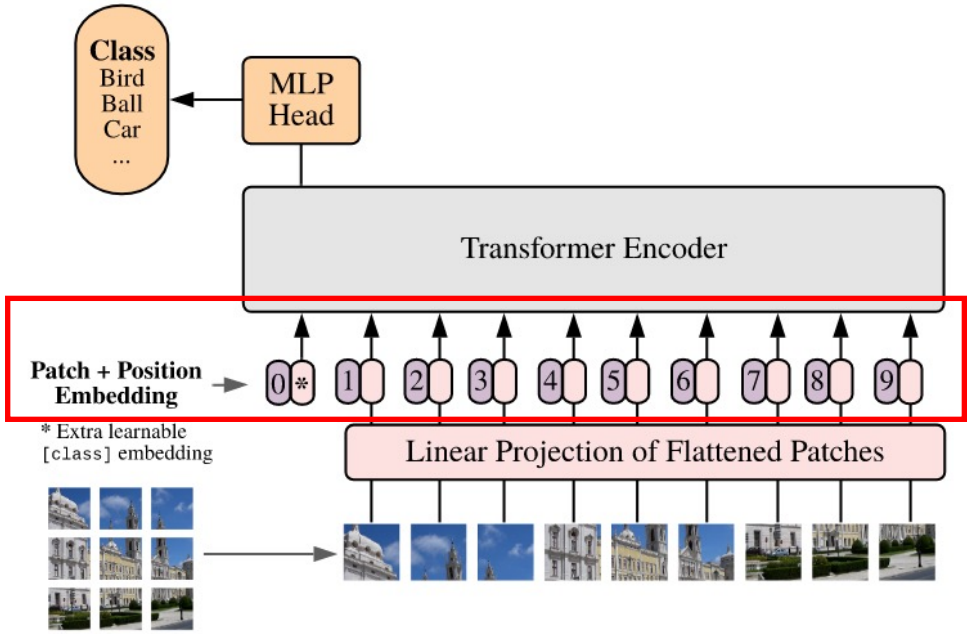


- 每一个token对应的输出表达了该token与其他部分之间的关联，具有偏向性，而Class token可以公平地获得全局分类信息
- Class token是一个额外的token，和token embedding输出相同

Transformer

❑ Vision Transformer (ViT)

✓ **An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. ICLR 2021.**



❑ Tokenization

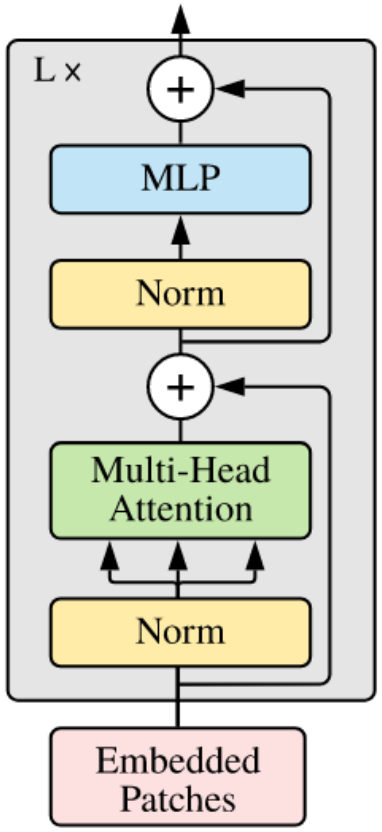
$$\mathbf{z}_0 = [\mathbf{x}_{class}; \mathbf{x}_p^1\mathbf{E}; \mathbf{x}_p^2\mathbf{E}; \cdots; \mathbf{x}_p^N\mathbf{E}] + \mathbf{E}_{pos}$$

其中$\mathbf{E} \in \mathbb{R}^{(P^2 \cdot C) \times D}$, $\mathbf{E}_{pos} \in \mathbb{R}^{(N+1) \times D}$

Vision Transformer

❑ Vision Transformer (ViT)

✓ 算法流程



❑ Transformer Encoder

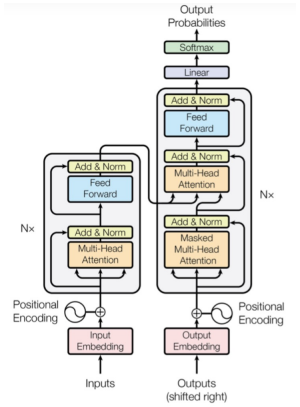
$$\mathbf{z}'_\ell = \mathrm{MSA}(\mathrm{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1}, \qquad \ell = 1 \ldots L$$
$$\mathbf{z}_\ell = \mathrm{MLP}(\mathrm{LN}(\mathbf{z}'_\ell)) + \mathbf{z}'_\ell, \qquad \ell = 1 \ldots L$$
$$\mathbf{y} = \mathrm{LN}(\mathbf{z}_L^0)$$

大纲

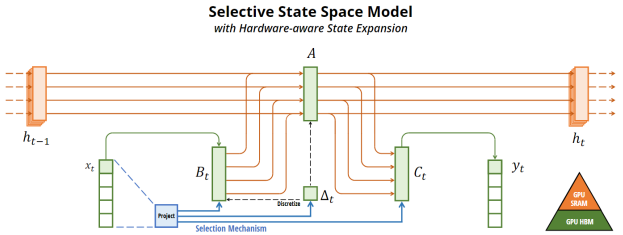
- ❑ 背景引入
- ❑ CNN基础知识
- ❑ 深度学习技巧
- ❑ 深度学习工具箱
- ❑ 经典网络架构
- ❑ Vision Transformer
- ❑ Mamba

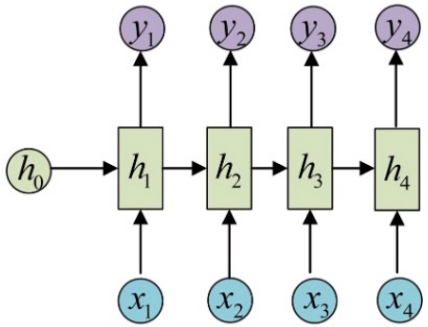
Mamba

1990年Jeffery L. Elman对Jodan Network进行简化得到了单节点RNN的原型

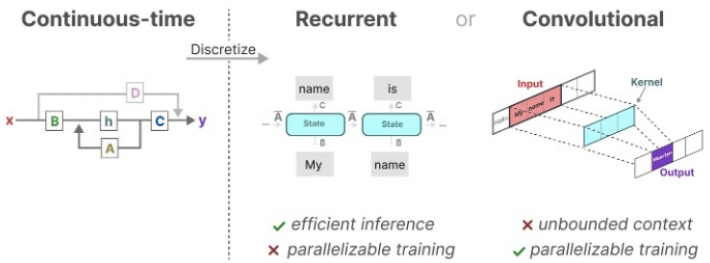


2021年Albert Gu提出S4模型，一种可以有效处理长序列的SSM





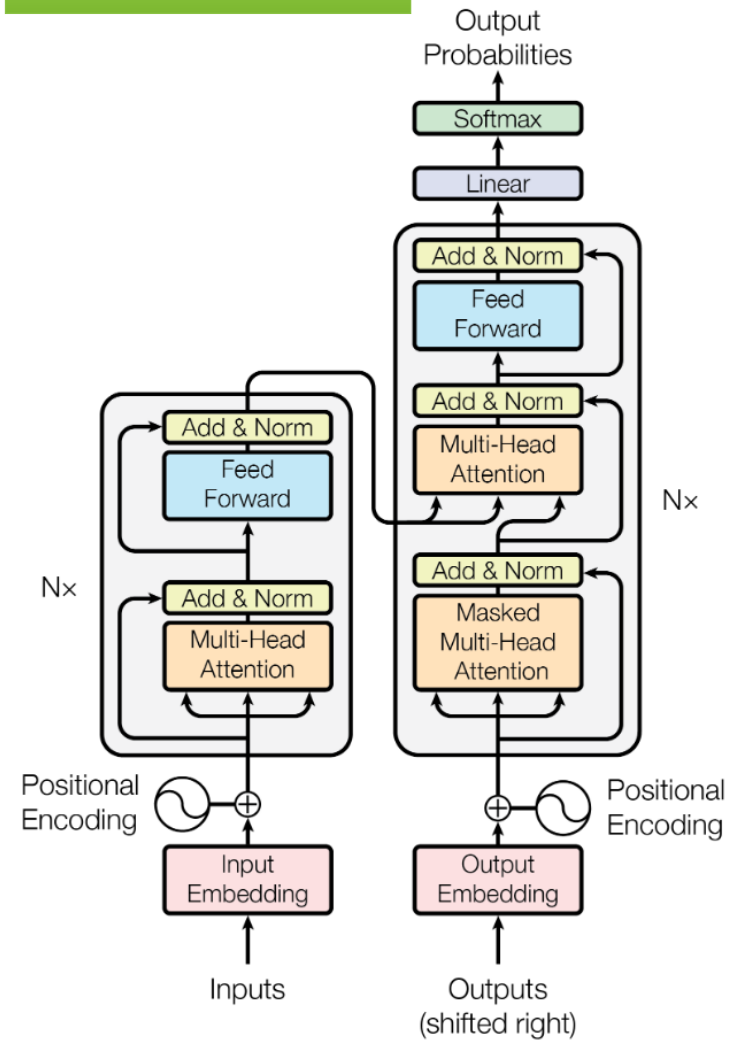
2017年Attention is all you need介绍了transformer架构开启了大模型时代

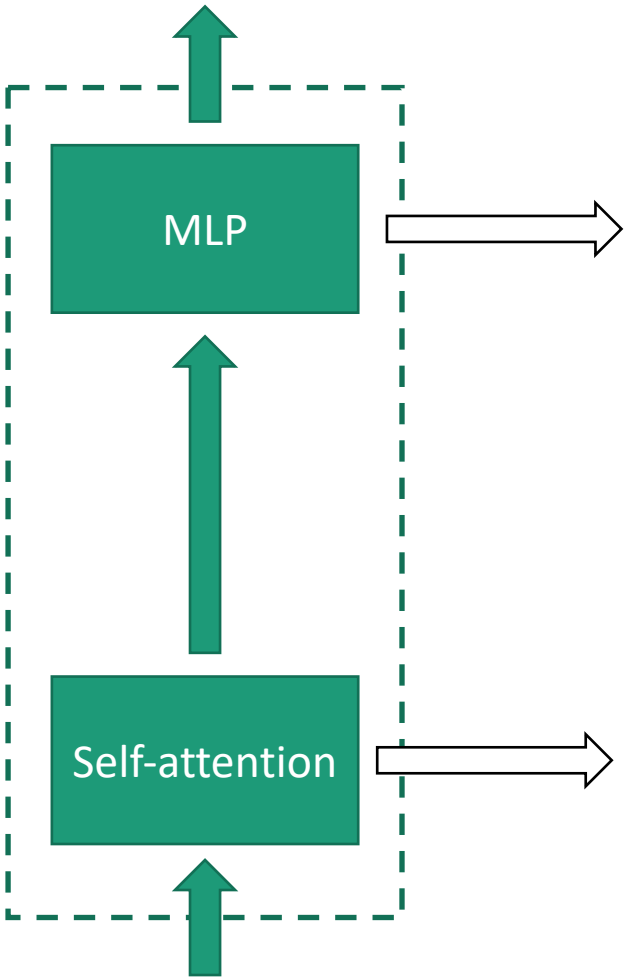


2023年Albert Gu提出Mamba模型从硬件访存的角度减少了运行时间

Mamba

计算复杂度





一般包含两个线性层第一层把维度从d映射到4d，第二层把维度从4d映射回d

总计算量 $= 16bNd^2$

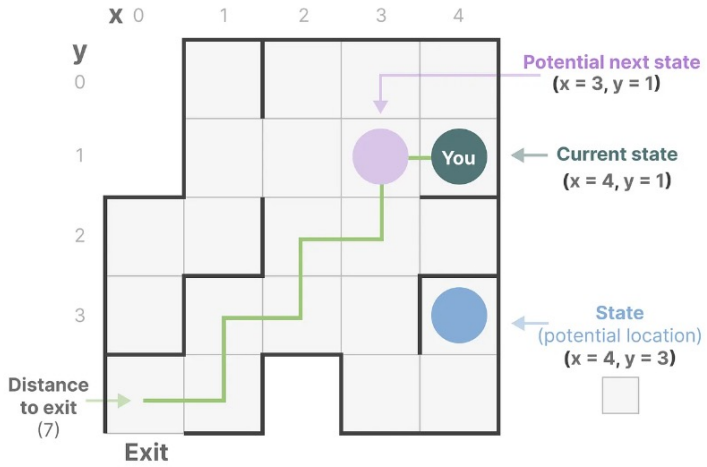
$$x_{\text{out}} = \text{softmax}(\frac{QK^T}{\sqrt{d}}) \cdot V \cdot W_o + x$$

总计算量 $= 8bNd^2 + 4bN^2d$

Mamba

状态空间

- 迷宫可以简化建模为一个“状态空间表示”
- 每一个小框显示你当前所在的位置，即当前状态 x_t
- 下一步可以去哪里， 即未来可能的状态 x_{t+1}
- 变化将你带到下一个状态，即当前执行的指令 u_t



描述状态的变量可以表示为“状态向量”

状态空间模型

SSM 是用于描述这些状态表示并根据某些输入预测其下一个状态可能是什么的模型，包括：

- 映射输入序列x(t)
- 潜在状态表示h(t)
- 预测输出序列y(t)

SSM不使用离散的数据输入，而是使用连续的序列

Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv 2023.

Mamba

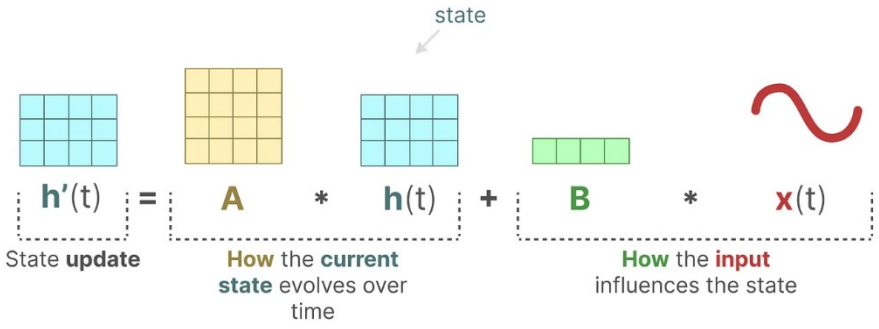
❑ SSM的两个方程（状态方程与输出方程）

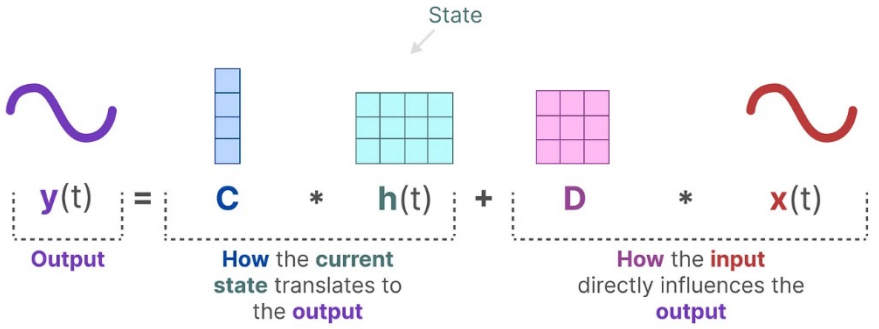
✓ 状态方程

- $h'(t) = \mathbf{A}h(t) + \mathbf{B}x(t)$
- B影响输入$x(t)$，A描述了所有内部状态如何连接

✓ 输出方程

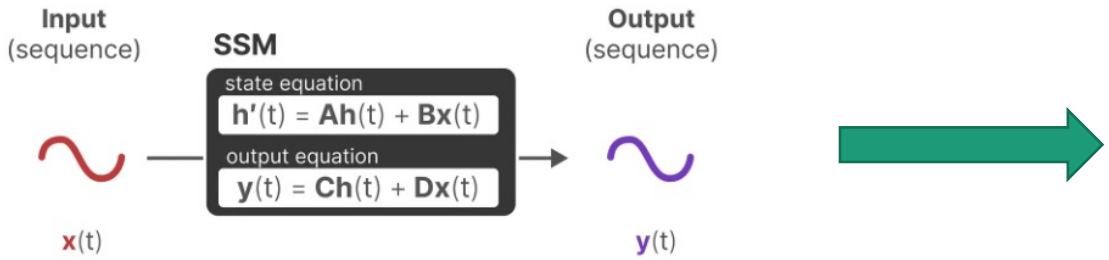
- $y(t) = \mathbf{C}h(t) + \mathbf{D}x(t)$
- 描述状态如何转换为输出

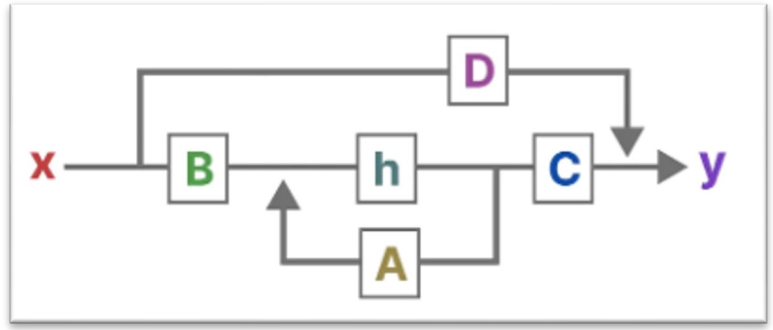




Mamba

❑ SSM模型

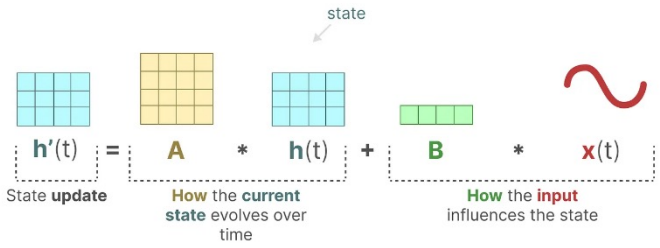




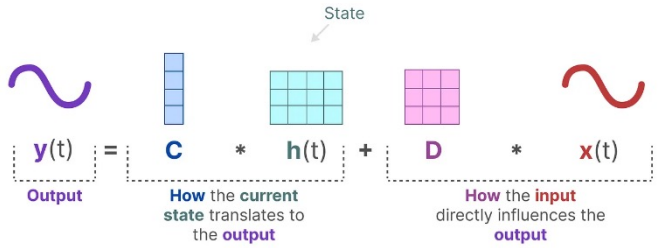
状态方程

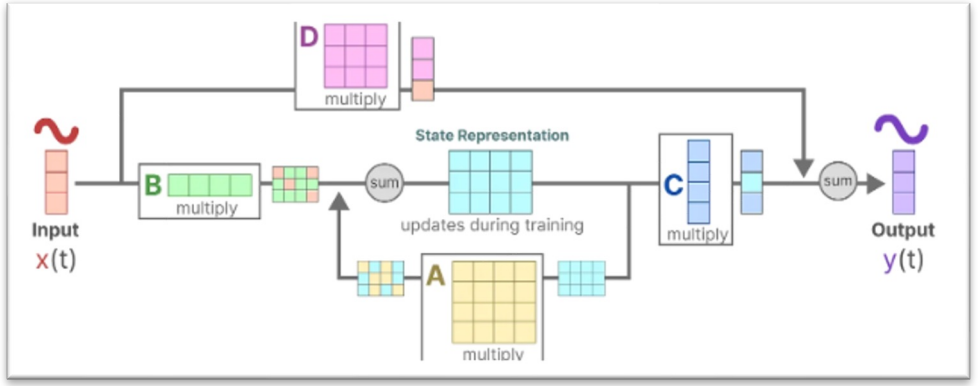
输出方程

- $h'(t) = \mathbf{A}h(t) + \mathbf{B}x(t)$
- B影响输入$x(t)$，A描述了所有内部状态如何连接



- $y(t) = \mathbf{C}h(t) + \mathbf{D}x(t)$
- 描述状态如何转换为输出



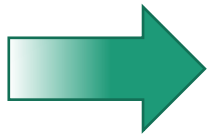


Mamba

□ 创新和对比

序列建模的一个基础问题是把上下文压缩成更小的状态:

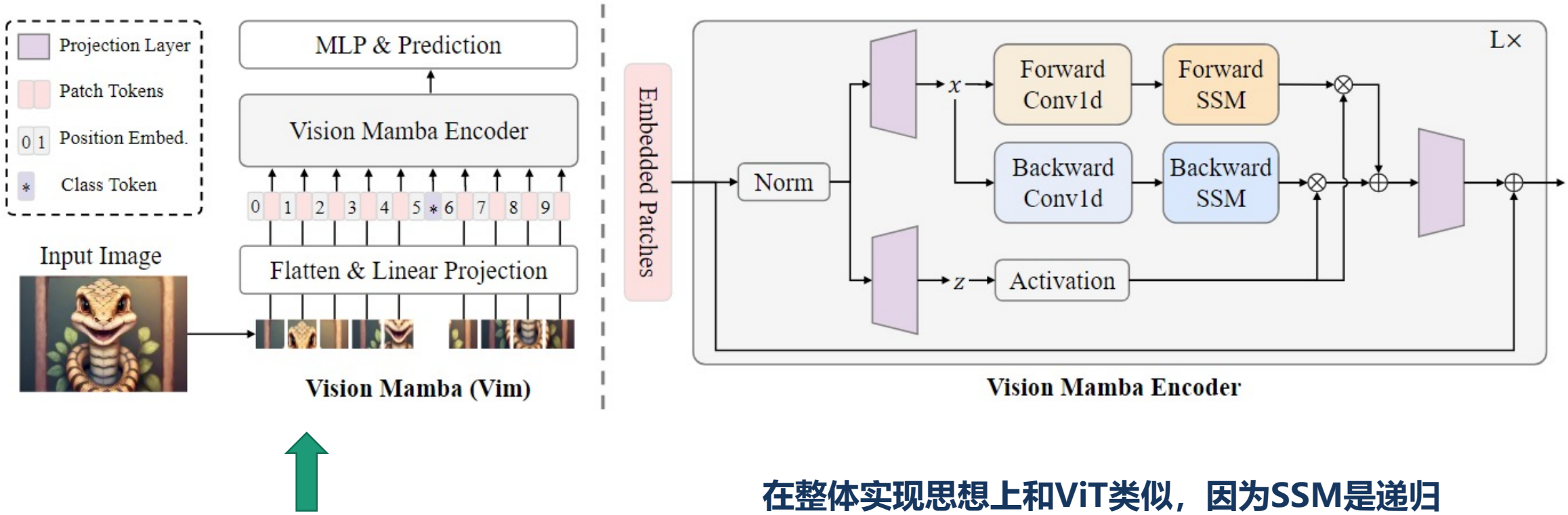
- □ 注意力机制:计算复杂度高，HBM存取代价大
- □ RNN:只能记住固定长度的上下文，无法并行训练
- □ SSM:其中的矩阵A、B、C始终是不变的，无法针对不同的输入针对性的推理



- □ 对输入信息有选择性处理
- □ 硬件感知算法
- □ 更简单的架构

模型	信息压缩程度	训练效率	推理效率
Transformer(注意力机制)	transformer对每个历史记录都不压缩	训练消耗算力大	推理消耗算力大
RNN	随着时间的推移，RNN 往往会忘记某一部分信息	RNN没法并行训练	推理时只看一个时间步 故推理高效(相当于推理快但训练慢)
CNN		训练效率高，可并行	
SSM	SSM压缩每一个历史记录		矩阵不因输入不同而不同，无法针对输入做针对性推理
mamba	选择性的关注必须关注的、过滤掉可以忽略的	mamba每次参考前面所有内容的一个概括，兼备训练、推理的效率	

Vision Mamba



使用一个卷积核大小为16*16*768，步长为16的卷积层来实现

在整体实现思想上和ViT类似，因为SSM是递归得到的，所以单个SSM只能考虑一个方向上的上下文，于是ViM设计了前向和后向两个SSM模块来获取全局上下文建模

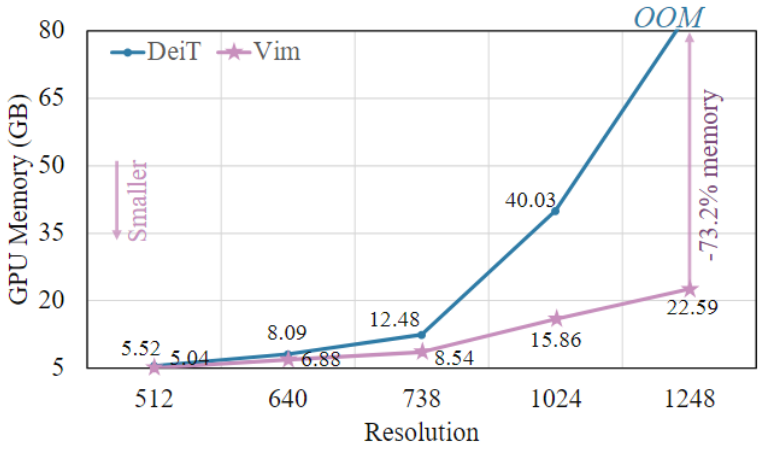
Vision Mamba: Efficient Visual Representation Learning with Bidirectional State Space Model. arXiv 2024.

Vision Mamba

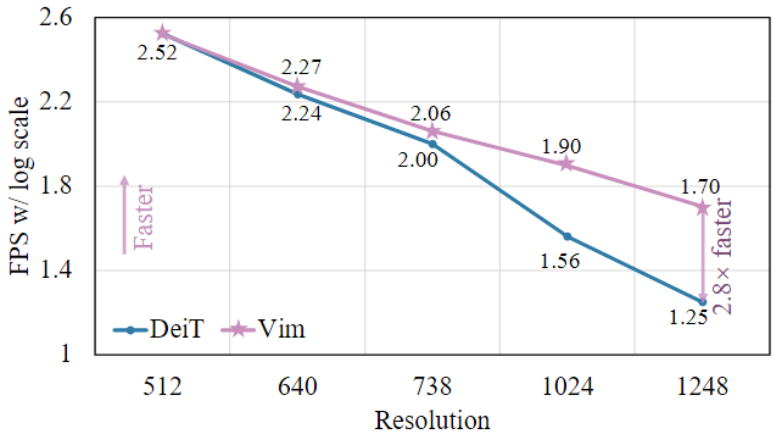
ImageNet分类性能

Method	image size	#param.	ImageNet top-1 acc.
Convnets			
ResNet-18	224^2	12M	69.8
ResNet-50	224^2	25M	76.2
ResNet-101	224^2	45M	77.4
ResNet-152	224^2	60M	78.3
ResNeXt50-32×4d	224^2	25M	77.6
RegNetY-4GF	224^2	21M	80.0
Transformers			
ViT-B/16	384^2	86M	77.9
ViT-L/16	384^2	307M	76.5
DeiT-Ti	224^2	6M	72.2
DeiT-S	224^2	22M	79.8
DeiT-B	224^2	86M	81.8
SSMs			
S4ND-ViT-B	224^2	89M	80.4
Vim-Ti	224^2	7M	76.1
Vim-Ti†	224^2	7M	78.3 +2.2
Vim-S	224^2	26M	80.5
Vim-S†	224^2	26M	81.6 +1.1

GPU占用量



推理速度



Vision Mamba: Efficient Visual Representation Learning with Bidirectional State Space Model. arXiv 2024.

南京大学
NANJING UNIVERSITY

谢 谢!

联系方式：liwenbin@nju.edu.cn

更多信息：https://cs.nju.edu.cn/liwenbin

神经网络作业

指导教师：李文斌

助教：陈怿陬

522023330016@smail.nju.edu.cn

2024年04月16日

作业要求

使用深度神经网络实现对图像数据集的分类

□ 数据集：Caltech 101

- 总共102个类别：包含101个物体类别，加上一个背景类别
- 包括人造物体和天然物体，例如蚂蚁、鲸鱼、灯泡、手风琴、飞机等
- 大多数类别约有40到800张图片，平均每个类别有50张图片
- 图片具有中等分辨率（约300x200像素）

数据集图像示例

作业要求

使用深度神经网络实现对图像数据集的分类

□ 模型：

- 可以使用一种任何你想使用的图像分类模型
- 包括但是不限于
  - **VGG**：[1409.1556] Very Deep Convolutional Networks for Large-Scale Image Recognition
  - **ResNet**：[1512.03385] Deep Residual Learning for Image Recognition
  - **ViT**：[2010.11929] An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale
  - 或者其他任何图像分类模型
- 可以使用官方代码实现

作业要求

使用深度神经网络实现对图像数据集的分类

□ 你需要进行如下探究

1. 使用你选择的模型实现对Caltech 101数据集的分类，你的模型在该数据集上的训练精确度和测试精确度如何
   - 绘制训练的训练误差和训练精度曲线
   - 并且汇报测试精确度

2. 尝试修改train_transform，添加更为先进的数据增广策略，验证其对分类效果的影响
   - Transforming and augmenting images — Torchvision 0.17 documentation (pytorch.org)

作业要求

使用深度神经网络实现对图像数据集的分类

口 你需要进行如下探究

3. 如果你使用的模型可以提供预训练参数，除了从零开始训练，另一种方式是微调预训练模型 （迁移学习）。
   - 尝试载入预训练参数，微调预训练模型，与从头训练相比，你有什么发现
   - 提示：微调预训练模型需要使用更小的学习率

作业要求

使用深度神经网络实现对图像数据集的分类

- **提交作业报告(.pdf)以及可运行代码文件(.py或.ipynb)，报告内容包括但不限于：**
  - 使用的分类模型，以及网络结构的描述。
  - 训练时候所使用的优化器，学习率，epoch数等细节信息。
  - 在Caltech 101数据集上的训练精确度曲线、训练损失曲线和测试精度。
  - 描述训练时数据增广策略是什么，使用此数据增广时，最终测试精度是多少？
  - 如果使用预训练的参数进行微调训练，可以达到的最大精确度是多少？你有什么发现？

- **可选内容（加分项）：**
  - 使用TSNE技术可视化训练样本的特征，可以任取10个类的样本进行可视化
  - 训练中遇到的任何异常情况和调试过程

作业步骤(简单示例)

0. 导入必要的包

```
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torchvision import models
from torch.utils.data import DataLoader, random_split
from torchvision import datasets, transforms
from tqdm import tqdm
```

注：所提供的代码仅为示例，可以任意修改

作业步骤(简单示例)

1. 首先下载数据集，并且定义训练和测试图像变换操作

```
    # Data augmentation for training and simple transforms for validation
    train_transform = transforms.Compose([
        transforms.RandomResizedCrop(224),
        # You can add more transform here...
        # ...
        transforms.ToTensor(),
        # If you use ImageNet1K pretrained-model, normalize as follow
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    ])

    test_transform = transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        # If you use ImageNet1K pretrained-model, normalize as follow
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    ])
```

作业步骤(简单示例)

2. 定义数据集和数据加载器

```
    # Define the dataset path
    dataset_path = '/XXXXXX/caltech-101/101_ObjectCategories/'
    dataset = datasets.ImageFolder(root=dataset_path, transform=train_transform)

    # Split dataset into training and testing
    train_size = int(0.8 * len(dataset))
    test_size = len(dataset) - train_size
    train_dataset, test_dataset = random_split(dataset, [train_size, test_size])

    # Apply different transforms to test dataset
    test_dataset.dataset.transform = test_transform

    # Create data loaders
    train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
    test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)
```

作业步骤(简单示例)

3. 定义模型：需要继承nn.Module并实现两个函数

- **__init__ :用于模型初始化**
- **forward：用于模型前向计算**

```
class SimpleCNN(nn.Module):
    def __init__(self, num_classes=102):
        super(SimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.conv3 = nn.Conv2d(in_channels=64, out_channels=128, kernel_size=3, padding=1)
        self.conv4 = nn.Conv2d(in_channels=128, out_channels=256, kernel_size=3, padding=1)
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)
        self.fc1 = nn.Linear(256 * 14 * 14, 1024)  # Adjusting for the flattened output size
        self.fc2 = nn.Linear(1024, num_classes)
```

注：此代码只是演示模型实现的例子，需要使用自己的模型

作业步骤(简单示例)

3. 定义模型：需要继承nn.Module并实现两个函数

- **__init__ :用于模型初始化**
- **forward：用于模型前向计算**

```
class SimpleCNN(nn.Module):
    def forward(self, x):
        # Apply the first convolution layer with ReLU activation function, then max pooling
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))
        x = self.pool2(F.relu(self.conv4(x)))
        # Flatten the output for the fully connected layer
        x = x.view(-1, 256 * 14 * 14)
        # Apply the fully connected layer with ReLU activation function
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

注：此代码只是演示模型实现的例子，需要使用自己的模型

作业步骤(简单示例)

4. 定义优化器与损失函数

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9, weight_decay=2e-4)
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=epochs)
```

5. 定义训练与测试流程

- 梯度清空：optimizer.zero_grad()
- 模型前向计算 outputs = model(images)
- 计算损失 loss = criterion(outputs, labels)
- 反向传播 loss.backward()
- 执行参数更新 optimizer.step()

作业步骤(简单示例)

5. 定义训练与测试流程

```python
def train_model(num_epochs, model, criterion, optimizer, scheduler,
                train_loader, device):
    model.train()
    for epoch in range(num_epochs):
        print(f'training on epoch:{epoch}')
        running_loss = 0.0
        total = 0
        correct = 0
        for images, labels in tqdm(train_loader):
            images, labels = images.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
            running_loss += loss.item() * images.size(0)
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
        scheduler.step()
        epoch_loss = running_loss / len(train_loader.dataset)
        epoch_accuracy = 100 * correct / total
        print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {epoch_loss:.4f}, Train Accuracy: {epoch_accuracy:.2f}%')
```

作业步骤(简单示例)

6. 主函数片段

```
    # Create data loaders
    train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
    test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

    # Instantiate the model
    model = SimpleCNN()

    # Move model to GPU if available
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    device = 'cpu'
    model.to(device)

    # Define loss function and optimizer
    epochs = 10
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9, weight_decay=2e-4)
    scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=epochs)

    # Run training and evaluation
    train_model(epochs, model, criterion, optimizer, scheduler, train_loader, device)
    evaluate_model(model, test_loader, device)
```

参考资料

- Pytorch教程：
  - Quickstart — PyTorch Tutorials
  - Transfer Learning for Computer Vision Tutorial

对抗学习

Adversarial Learning

助教：赖伟

522023330042@smail.nju.edu.cn

2024年04月16日

目录

- 什么是对抗学习
- 对抗攻击
- 对抗防御
- 其他内容

目录

什么是对抗学习

- 背景
  - 深度学习在实际应用中具有优越的性能。
  - 在真实数据上表现良好的深度学习模型很容易被攻击者恶意构造的对抗样本（adversarial example）欺骗。
  - 生成对抗样本的行为被称为对抗攻击，防御对抗样本的行为被称为对抗防御。
- 意义
  - 人工智能技术在军事、工业、金融、医疗、人脸识别、自动驾驶等不同领域中被广泛应用
  - 对抗样本带来的威胁愈发突出
  - 影响人们对人工智能的信任

- 原始图像->bird: 90%

对抗攻击

- 对抗样本->dog: 95%
  - 肉眼很难看出区别
  - 能够欺骗模型

目录

- 什么是对抗学习
- 对抗攻击
- 对抗防御
- 其他内容

对抗攻击

❑ 对抗样本

- 向原始样本中添加微小的扰动得到
- 人眼无法看出二者区别
- 模型会产生不同的预测

$$C(x^*) \neq y, \qquad s.t.\ \ D(x^*, x) \leq \epsilon$$

❑ 对抗攻击的分类

- 目标不同：
  - 有目标攻击 → 指定攻击目标y^*，要求$C(x^*) = y^*$
  - 无目标攻击 → 无特定攻击目标，要求$C(x^*) \neq y$
- 攻击者知识不同：
  - 白盒攻击 → 攻击者完全知道模型的梯度参数等信息
  - 黑盒攻击 → 攻击者不知道模型的详细信息

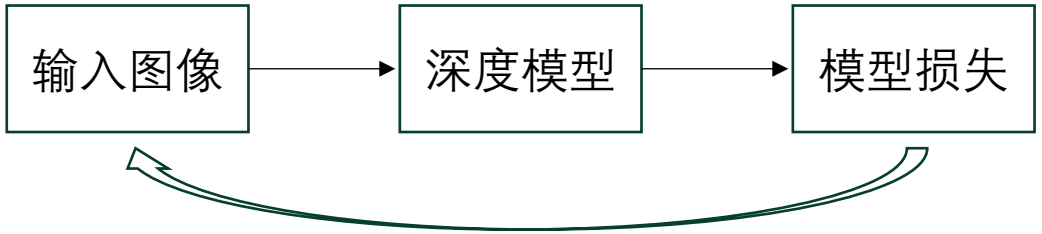
对抗攻击

❑ 白盒攻击

- 模型参数完全公开
- 攻击者依靠模型的梯度优化输入图像
- 攻击性更强，成功率更高

神经网络训练过程

损失最小化反向传播



对抗攻击过程

损失最大化反向传播

对抗攻击

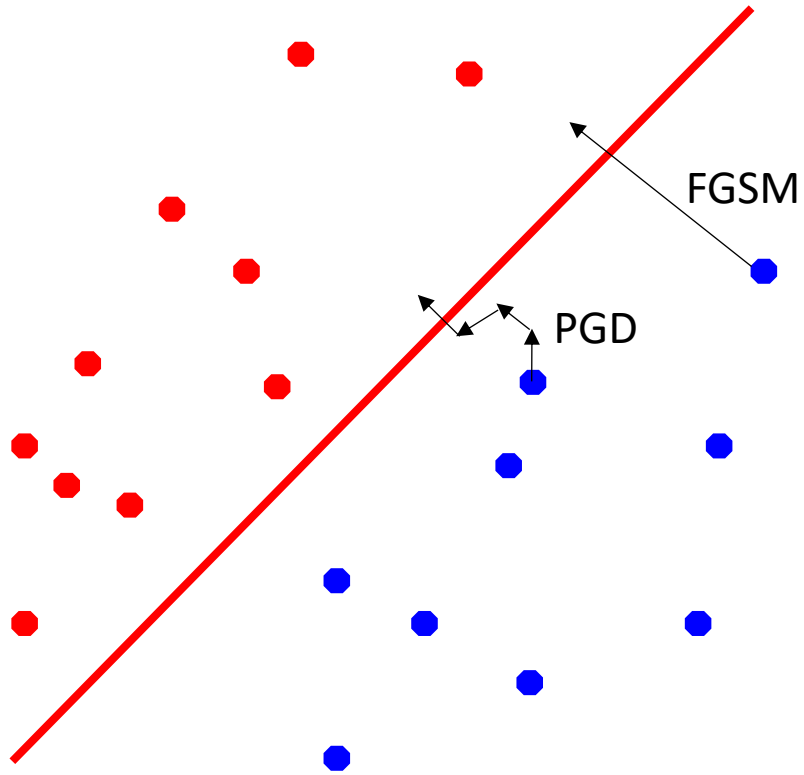
❑ 白盒攻击

- 快速梯度符号法：FGSM

$$x_{adv} = x + \epsilon \cdot \text{sign}\left(\nabla_x L_\theta(x, y)\right)$$

- 投影梯度法：PGD

$$x_{\text{adv}}^{t+1} = \Pi_S\left(x_{\text{adv}}^{t} + \alpha \cdot \text{sign}\left(\nabla_{x_{\text{adv}}} L_\theta\left(x_{\text{adv}}^{t}, y\right)\right)\right)$$



- Goodfellow I J, Shlens J, Szegedy C. Explaining and harnessing adversarial examples[J]. arXiv preprint arXiv:1412.6572, 2014.
- Madry A, Makelov A, Schmidt L, et al. Towards deep learning models resistant to adversarial attacks[J]. arXiv preprint arXiv:1706.06083, 2017.

对抗攻击

- **白盒攻击**
  - C&W攻击
  - JSMA攻击
  - DeepFool
  - AutoAttack
  - 。。。

- Carlini N, Wagner D. Towards evaluating the robustness of neural networks[C]//2017 ieee symposium on security and privacy (sp). Ieee, 2017: 39-57.
- Papernot N, McDaniel P, Jha S, et al. The limitations of deep learning in adversarial settings[C]//2016 IEEE European symposium on security and privacy (EuroS&P). IEEE, 2016: 372-387.
- Moosavi-Dezfooli S M, Fawzi A, Frossard P. Deepfool: a simple and accurate method to fool deep neural networks[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2016: 2574-2582.
- Croce F, Hein M. Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks[C]//International conference on machine learning. PMLR, 2020: 2206-2216.

对抗攻击

❑ 对抗攻击参数

- 生成对抗样本的迭代次数
- 每次迭代的步长
- 最大累计迭代距离
- 扰动范数：L_2和L_∞
- 举例: PGD(10, 2/255, 8/255)

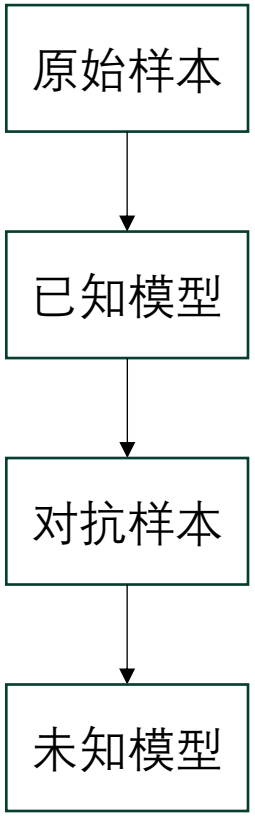
对抗攻击

❑ 黑盒攻击

- 更适合现实场景
- 实际应用范围更广泛
- 成功率相对较低

❑ 攻击方式

- 黑盒迁移攻击
- 黑盒得分攻击
- 黑盒决策攻击



对抗攻击

❑ 物理攻击

- 以往攻击往往存在于数字世界中
- 针对真实场景的攻击
- 环境变化、空间相对位置变化、对抗扰动的物理可实现性等挑战

❑ 攻击领域

- 人脸识别
- 路牌识别
- 行人检测

Sharif M, Bhagavatula S, Bauer L, et al. Accessorize to a crime: Real and stealthy attacks on state-of-the-art face recognition[C]//Proceedings of the 2016 acm sigsac conference on computer and communications security. 2016: 1528-1540.

目录

- 什么是对抗学习
- 对抗攻击
- 对抗防御
- 其他内容

对抗防御

❑ 定义

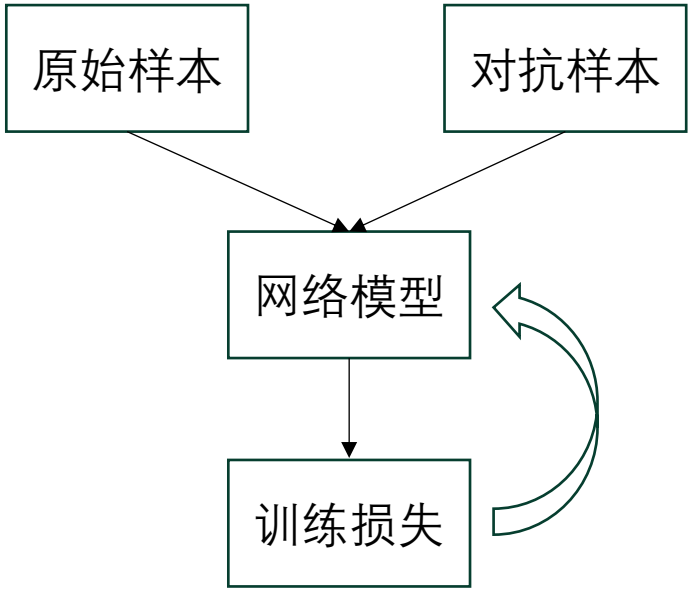
- 通过各种方式使对抗攻击无法成功

❑ 常用手段

- 对抗训练
- 图像变换
- 随机化

对抗防御

- **对抗训练：**
  - 在训练时加入对抗样本一起训练
  - 目前最有效的对抗防御方法
  - 计算成本比较高
  - 降低模型在干净样本上的准确率
  - 干净准确率和鲁棒准确率之间存在权衡(trade-off)



对抗防御

❑ Adversarial Training：

- 首先提出对抗训练
- 使用对抗样本代替原始样本进行训练

$$\min_{\theta} E_{(x,y)\sim\mathcal{D}}\left[\max_{\|\delta\|_\infty<\epsilon} L_\theta(x+\delta,y)\right]$$

❑ TRADES:

- 直接使用对抗样本训练会损害在干净样本上的精度
- 使用干净样本和对抗样本的输出差异作为正则项
- 兼顾精度与鲁棒性

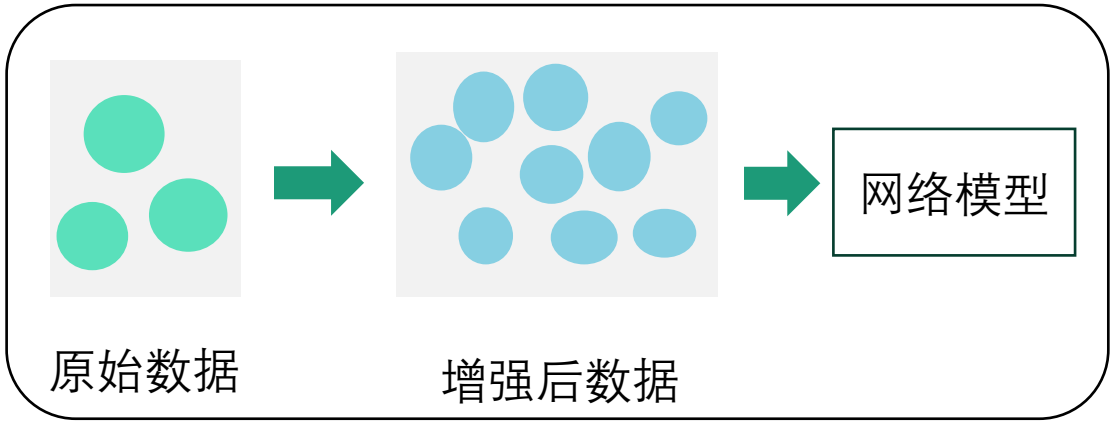
$$\min_f \mathbb{E}\Big\{\underbrace{\phi(f(\boldsymbol{X})Y)}_{\text{for accuracy}}+\underbrace{\max_{\boldsymbol{X}'\in\mathbb{B}(\boldsymbol{X},\epsilon)}\phi(f(\boldsymbol{X})f(\boldsymbol{X}')/\lambda)}_{\text{regularization for robustness}}\Big\}.$$

- Madry A, Makelov A, Schmidt L, et al. Towards deep learning models resistant to adversarial attacks[J]. arXiv preprint arXiv:1706.06083, 2017.
- Zhang H, Yu Y, Jiao J, et al. Theoretically principled trade-off between robustness and accuracy[C]//International conference on machine learning. PMLR, 2019: 7472-7482.

对抗防御

❑ 数据增强：

- 使用多样化的数据增强能提升模型鲁棒性
- 包括人工设计的增强和生成模型生成的新样本
- 本质是提升模型的泛化性



❑ 分离对抗特征和干净特征：

- 将对抗样本的特征视为对抗特征和干净特征的结合
- 把两类特征分开处理，能取得更好的效果

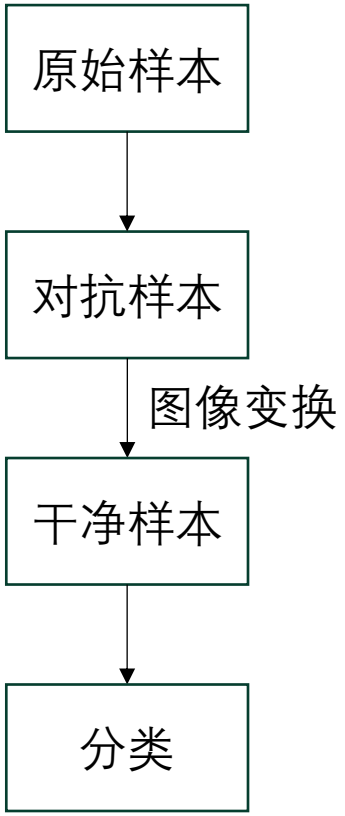
Rebuffi S A, Gowal S, Calian D A, et al. Fixing data augmentation to improve adversarial robustness[J]. arXiv preprint arXiv:2103.01946, 2021.
Kim W J, Cho Y, Jung J, et al. Feature separation and recalibration for adversarial robustness[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2023: 8183-8192.

对抗防御

- 知识蒸馏
- 对抗训练的攻击策略选取
- 对抗训练正则项设计
- 。。。

对抗防御

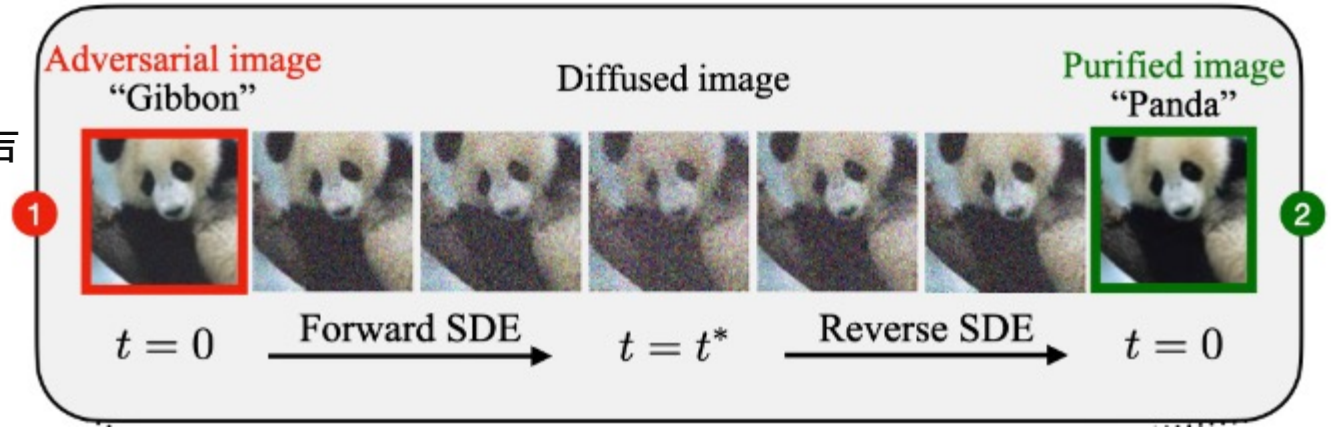
- **图像变换：**
  - 对输入的样本或是特征进行重建
  - 使重建后的样本尽量逼近原始干净样本
  - 目标是消除对抗扰动对样本的影响
  - 本质是一个净化的过程



对抗防御

❑ 特征去噪：

- 对抗样本与原始样本在特征层面有明显噪声
- 使用去噪技术对特征进行净化



❑ 图像重建：

- 现在的生成模型非常强大，能生成高质量的样本
- 利用生成模型将对抗样本变成干净样本

Xie C, Wu Y, Maaten L, et al. Feature denoising for improving adversarial robustness[C]//Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2019: 501-509.

Lin G, Li C, Zhang J, et al. Adversarial Training on Purification (AToP): Advancing Both Robustness and Generalization[J]. arXiv preprint arXiv:2401.16352, 2024.

对抗防御

❑ 随机化：

- 向输入数据或深度学习模型中加入随机性的防御。
- 使得攻击目标函数对于输入数据的梯度具有随机性，防止基于梯度的对抗攻击生成对抗样本。

Pang T, Xu K, Zhu J. Mixup inference: Better exploiting mixup to defend adversarial attacks[C]//International Conference on Learning Representations. 2020.
Xie C, Wang J, Zhang Z, et al. Mitigating adversarial effects through randomization[C]// International Conference on Learning Representations. 2018.

目录

- 什么是对抗学习
- 对抗攻击
- 对抗防御
- 其他内容

Leaderboards

- **RobustBench：**
  - 提供一个通用公平的对抗防御算法对比benchmark
  - 收录多种对抗防御方法，方便快速了解当前对抗防御的进展
  - 也提供了model zoo，方便调用多种对抗防御算法

Croce F, Andriushchenko M, Sehwag V, et al. Robustbench: a standardized adversarial robustness benchmark[J]. arXiv preprint arXiv:2010.09670, 2020.

Leaderboards

RobustBench Leaderboards Paper FAQ Contribute Model Zoo

Leaderboard: CIFAR-100, $\ell_\infty = 8/255$, untargeted attack

Show 15 entries Search: Papers, architectures, ve

Rank	Method	Standard accuracy	AutoAttack robust accuracy	Best known robust accuracy	AA eval. potentially unreliable	Extra data	Architecture	Venue
1	Better Diffusion Models Further Improve Adversarial Training *It uses additional 50M synthetic images in training.*	75.22%	42.67%	42.67%	✗	✗	WideResNet-70-16	ICML 2023
2	MixedNUTS: Training-Free Accuracy-Robustness Balance via Nonlinearly Mixed Classifiers *It uses an ensemble of networks. The robust base classifier uses 50M synthetic images. 41.80% robust accuracy is due to the original evaluation (Adaptive AutoAttack)*	83.08%	41.91%	41.80%	✗	☑	ResNet-152 + WideResNet-70-16	arXiv, Feb 2024
3	Decoupled Kullback-Leibler Divergence Loss *It uses additional 50M synthetic images in training.*	73.85%	39.18%	39.18%	✗	✗	WideResNet-28-10	arXiv, May 2023
4	Better Diffusion Models Further Improve Adversarial Training *It uses additional 50M synthetic images in training.*	72.58%	38.83%	38.83%	✗	✗	WideResNet-28-10	ICML 2023

https://robustbench.github.io/

Croce F, Andriushchenko M, Sehwag V, et al. Robustbench: a standardized adversarial robustness benchmark[J]. arXiv preprint arXiv:2010.09670, 2020.

常用的算法库

- AdverTorch：https://github.com/BorealisAI/advertorch
- CleverHans：https://github.com/cleverhans-lab/cleverhans
- DeepRobust：https://github.com/DSE-MSU/DeepRobust
- ARES：https://github.com/thu-ml/ares

常用的算法库

```python
from advertorch.attacks import LinfPGDAttack
Codeium: Refactor | Explain | Generate Docstring | ×
def evaluate_adversarial(model, adversary, data_loader, device="cuda"):
    model.eval()  # 将模型设置为评估模式
    correct = 0
    adv_correct = 0
    total = 0
    adversary = LinfPGDAttack(
        model,
        loss_fn=nn.CrossEntropyLoss(reduction="mean"),
        eps=8 / 255,
        nb_iter=10,
        eps_iter=2 / 255,
        rand_init=True,
        clip_min=0.0,
        clip_max=1.0,
        targeted=False,
    )

    for inputs, labels in data_loader:
        inputs, labels = inputs.to(device), labels.to(device)
        adv_inputs = adversary.perturb(inputs, labels)  # 生成对抗样本

        with torch.no_grad():  # 在评估过程中不计算梯度
            adv_outputs = model(adv_inputs)
            outputs = model(inputs)
            _, adv_predicted = torch.max(adv_outputs.data, 1)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            adv_correct += (adv_predicted == labels).sum().item()
            correct += (predicted == labels).sum().item()

    adv_accuracy = adv_correct / total
    accuracy = correct / total
    return accuracy, adv_accuracy
```

南京大学
NANJING UNIVERSITY

谢 谢!